

Documentación proyecto de título

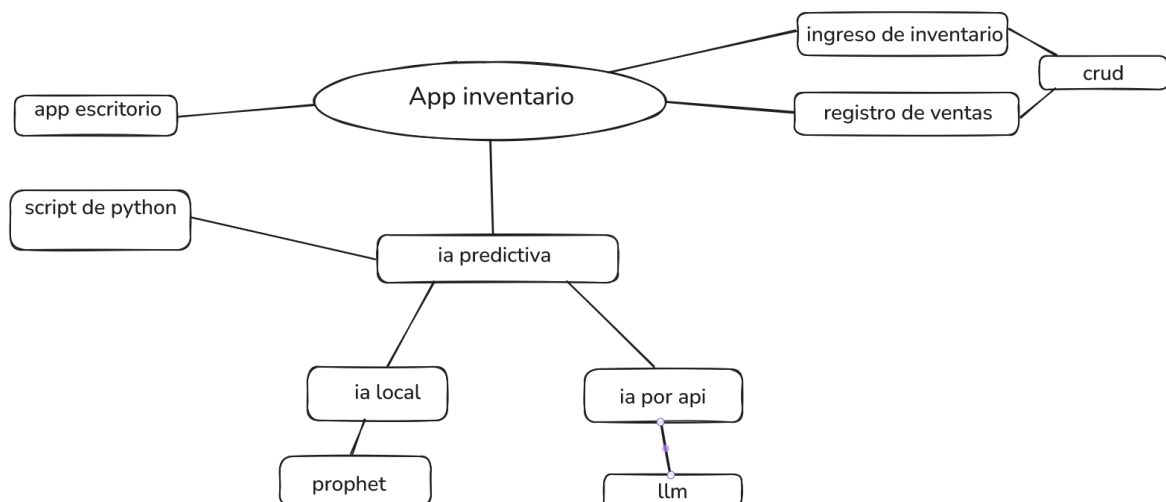
Integrantes: Yerko Navarro, Isaac González, Almendra Pereira.

Profesor: Nancy Bernal

Análisis del caso/Contexto

Nos contactamos con la cafetería Eluney para ofrecer nuestro servicio como desarrolladores, el dueño nos comentó que presenta problemas a la hora de gestionar su inventario y le gustaría tener más alcance en su proyecto ya que según sus propias palabras desea duplicar la venta. La falta de stock en inventario o el sobrestock afectan a las ganancias. Es por ello que le sugerimos **desarrollar un software** para poder gestionar inventario y una **página web** publicitaria para expandir el negocio.

Mapa mental



Visión del Proyecto

"Transformar la gestión operativa de la tienda mediante un ecosistema digital integrado que garantice el control total del inventario en tiempo real, agilice el proceso de ventas en el punto físico y proyecte una imagen moderna y accesible hacia los clientes a través de una carta digital siempre actualizada."

Pilares fundamentales

1. Experiencia de Usuario Omnicanal (UX/UI)

- **Enfoque:** Garantizar que tanto el cliente externo como el personal interno (vendedor/administrador) tengan una interfaz intuitiva.
- **Justificación Técnica:** Implementación de diseño *Mobile-First* y navegación simplificada para asegurar que el proceso de compra y venta sea fluido en PC y dispositivos móviles.

2. Integridad y Trazabilidad de Inventario

- **Enfoque:** Control estricto de entradas, salidas, ajustes por pérdida y anulaciones de ventas.
- **Justificación Técnica:** Registro de auditoría (logs) para cada movimiento manual y actualización automática de stock en tiempo real tras cada transacción o anulación.

3. Seguridad y Jerarquía de Acceso (RBAC)

- **Enfoque:** Separación de funciones mediante roles (Administrador y Vendedor) y protección de datos.
- **Justificación Técnica:** Implementación de niveles de acceso (Role-Based Access Control) y cifrado de credenciales para prevenir la fuga de información sensible o modificaciones no autorizadas.

4. Continuidad Operativa y Analítica

- **Enfoque:** Protección contra pérdida de datos (respaldos)
- **Justificación Técnica:** Automatización de copias de seguridad de la base de datos.

Historia de usuario

Estimación de esfuerzo: 1,2,3,5,8,13,21

1. Como un cliente quiero ver una página intuitiva, con navegación fácil y que funcione el pc y teléfono (5)
2. Como gestor necesito saber un balance de mi inventario, donde rápidamente puede ver que se está agotando (8)
3. Como vendedor necesito registrar las ventas del día, donde pueda agregar la orden de pedido, el precio, y un resumen de cuanto se ha vendido en el día (3)
4. Como dueño del negocio me gustaría tener un gráfico en donde pueda hacer un análisis de cómo va mi negocio(13)
5. Como dueño del negocio necesito que mis clientes puedan acceder a mis redes sociales desde la página web, y un apartado de contacto. (3)
6. Como Administrador quiero poder crear, editar y eliminar un producto.(3)
7. Como administrador quiero que el sistema tenga roles y contraseñas para que un vendedor no pueda eliminar inventario o ver datos sensibles del negocio.(13)
8. Como administrador quiero que el sistema haga respaldos automáticos de la base de datos para no perder información. (8)
9. Como administrador quiero registrar la entrada de mercadería. (3)
10. Como administrador quiero realizar ajustes manuales de stock por pérdida, daño u otros motivos dejando una nota. (3)
11. Como Administrador, quiero poder anular una venta realizada por error para que el stock regrese al inventario y el dinero se descuenta del total. (3)

Requisitos del Sistema

1. Gestión de Usuarios y Seguridad

Requisitos Funcionales:

- **RF01:** El sistema debe implementar **Control de Acceso Basado en Roles (RBAC)** con al menos dos niveles: Administrador y Vendedor.
- **RF02:** El sistema debe restringir el acceso del "Vendedor" a módulos de costos, utilidades netas y edición directa de inventario.
- **RF03:** El sistema debe contar con un módulo de autenticación (Login) mediante usuario y contraseña.

Requisitos No Funcionales:

- **RNF01 (Seguridad):** Las contraseñas deben ser almacenadas mediante algoritmos de hash (ej. BCrypt) y no en texto plano.
 - **RNF02 (Confidencialidad):** Los datos sensibles del negocio (márgenes de ganancia) deben estar encriptados o bloqueados para roles no autorizados.
-

2. Catálogo e Inventario

Requisitos Funcionales:

- **RF04:** El administrador debe poder realizar el **CRUD completo** (Crear, Leer, Actualizar, Eliminar) de productos.
- **RF05:** El sistema debe permitir el registro de entrada de mercadería para aumentar el stock disponible.
- **RF06:** El sistema debe permitir **ajustes manuales de stock** con un campo obligatorio de "Nota de Motivo" (daño, pérdida, vencimiento).
- **RF07:** El sistema debe mostrar alertas visuales (Dashboard) de productos con **stock bajo** o agotándose.

Requisitos No Funcionales:

- **RNF03 (Integridad):** El sistema no debe permitir la eliminación física de un producto si tiene historial de ventas (borrado lógico).
 - **RNF04 (Trazabilidad):** Cada ajuste manual debe registrar el usuario, la fecha y el motivo en un log de auditoría.
-

3. Ventas y Punto de Venta

Requisitos Funcionales:

- **RF08:** El vendedor debe poder registrar ventas, asociando productos, precios y generando una orden de pedido.
- **RF09: (Anulación)** El administrador debe poder anular una venta errónea, lo cual debe reintegrar automáticamente el stock al inventario y descontar el dinero del total diario.
- **RF10 (Opcional):** El sistema debe permitir el escaneo de productos mediante código de barras.

Requisitos No Funcionales:

- **RNF05 (Performance):** El sistema debe validar el stock en tiempo real en máximo 5 segundos
-

4. Interfaz de Cliente y Presencia Digital (Web)

Requisitos Funcionales:

- **RF12:** El sistema debe mostrar iconos interactivos de redes sociales y una sección de contacto.
- **RF13:** El formulario de contacto debe enviar una notificación al administrador tras ser completado.

Requisitos No Funcionales:

- **RNF06 (Responsividad):** La interfaz debe ser **100% Mobile-First**, adaptándose a PC y teléfonos de forma fluida.
 - **RNF07 (Usabilidad):** El menú de navegación debe ser jerárquico y simple (máximo 3 niveles de profundidad).
-

5. Análisis y Respaldo

Requisitos Funcionales:

- **RF15:** El sistema debe generar **gráficos analíticos** (ej: ventas vs tiempo, productos más vendidos) accesibles solo para el Dueño.
- **RF16:** El sistema debe realizar **respaldos automáticos** de la base de datos (backups diarios).
- **RF17:** El sistema debe mostrar un resumen diario de ventas totales.

Requisitos No Funcionales:

- **RNF08 (Disponibilidad):** El sistema debe estar disponible el 99.5% del tiempo.
- **RNF09 (Recuperación):** En caso de falla crítica, los respaldos deben permitir la restauración total de los datos en menos de 2 horas.

6. Definición de responsabilidades RACI

- **Responsive:** UI y desarrollo frontend (Almendra) Integración AI (Yerko) Lógica backend(Isaac)
- **Accountable:** Profesora Nancy Bernal
- **Consulted:** Almendra, Yerko, Isaac
- **Informed:** Cliente(Alex Carcey)

Matriz de Riesgos

-R01 - Falla en la IA Predictiva (Prophet/LLM): Este riesgo presenta una probabilidad **Media** y un impacto **Alto**, resultando en un nivel **Crítico**. El evento implica que las predicciones de stock sean erróneas, lo que causaría sobrestock o falta de productos, afectando directamente las ganancias. La estrategia es mitigar mediante pruebas exhaustivas con datos históricos de la cafetería antes de la puesta en marcha oficial.

-R02 - Dependencia de Conectividad para IA (API): Con una probabilidad **Media** y un impacto **Alto**, este riesgo se sitúa en un nivel **Alto**. Consiste en la posible indisponibilidad de la API de IA o falta de internet, lo que impediría generar análisis en tiempo real. El plan de acción es mitigar implementando un modo de respaldo que utilice la IA local (Prophet) cuando la conexión externa falle.

-R03 - Brecha de Seguridad en Accesos: Este riesgo tiene una probabilidad **Baja** pero un impacto **Alto**, con un nivel de riesgo **Alto**. Se refiere a que un usuario con rol de vendedor acceda a datos sensibles (como costos o utilidades) o elimine inventario sin permiso. La estrategia es evitar mediante la implementación estricta de RBAC y el cifrado de contraseñas con BCrypt.

-R04 - Pérdida Crítica de Datos: Presenta una probabilidad **Baja** y un impacto **Catastrófico**, manteniendo un nivel **Alto**. Implica una falla del servidor que resulte en la pérdida del inventario o registros de venta. El plan es mitigar automatizando los backups diarios y garantizando una restauración total en menos de 2 horas.

-R05 - Incompatibilidad en Experiencia Omnicanal (Web): Con probabilidad **Media** e impacto **Medio**, el nivel es **Medio**. El riesgo es que la interfaz no sea totalmente Mobile-First, dificultando la navegación en los teléfonos de los clientes. La acción consiste en mitigar a través de pruebas de usabilidad en múltiples dispositivos y navegadores antes del lanzamiento.

-R06 - Resistencia al Uso del Sistema: Este riesgo tiene una probabilidad **Alta** y un impacto **Medio**, resultando en un nivel **Medio**. Se refiere a que el personal de la cafetería encuentre el software difícil de operar en el día a día. La estrategia es Mitigar diseñando una interfaz intuitiva (UX/UI) y realizando sesiones de capacitación técnica al personal.

EDT Hitos de desarrollo y diccionario

Desglose de trabajo:

Nivel 1 : Sistema de ventas y sitio web.

Nivel 2: Aplicación de escritorio, sitio web con carta, base de datos, documentación.

Nivel 3: Módulo ventas, módulo de gestión de usuarios, módulo de inventario, módulo de estadísticas, módulo de gestión de productos.

Diccionario de la edt

Entregables:

1- Aplicación de escritorio: Módulo de ventas

- Descripción: Desarrollo de la interfaz gráfica hasta la lógica necesaria del módulo. Debe ser posible registrar una venta, cancelar una venta, calcular el total de productos con IVA incluido y también debe ser posible realizar un descuento.
- La interfaz debe reaccionar ante errores en el llenado de campos, como espacios en blancos, formato equivocado y números negativos .
- Responsable: Desarrollador backend.
- Criterios de aceptación: 1- El cálculo total debe ser exacto y en el formato de peso chileno.
- Al registrar o cocinar platillos debe descontarse automáticamente la cantidad del inventario.
- El sistema debe impedir ventas si el stock está en cero.
- El tiempo de la transacción debe ser máximo 2 segundos.

2- Módulo gestión de usuarios:

- Descripción: Desarrollo de la lógica que permita tener usuarios con distintos roles.
- Responsable: Desarrollador backend.
- Criterios de aceptación: El sistema debe permitir el ingreso solo a usuarios registrados con un nombre de usuario y contraseña correctos.
- Si las credenciales son incorrectas, el sistema debe mostrar un mensaje genérico ("Usuario o contraseña incorrectos") sin especificar cuál de los dos falló (por seguridad).
-

- El administrador debe poder resetear la contraseña de cualquier usuario en caso de olvido.
- **Restricción de Interfaz:** Si un usuario tiene el rol "**Vendedor**", los botones o menús de "Configuración de Sistema", "Costos" y "Reportes de Utilidad" deben estar ocultos o deshabilitados.
- **Permisos de Acción:** El sistema debe impedir que un usuario con rol "**Vendedor**" elimine registros de inventario o anule ventas (esta acción debe requerir la clave de un **Administrador**).
- **Acceso Total:** El usuario con rol "**Administrador**" debe tener acceso sin restricciones a todos los módulos (Escritorio y configuración Web).
- **Cifrado de Contraseñas:** Las contraseñas **no deben guardarse en texto plano** en la base de datos. Se debe utilizar un algoritmo de hashing seguro (ej: BCrypt o Argon2).
- **Sesiones Activas:** El sistema debe cerrar la sesión automáticamente tras un periodo de inactividad (ej. 30 minutos) para evitar accesos no autorizados en la caja física.
- **Unicidad:** El sistema no debe permitir la creación de dos usuarios con el mismo nombre de usuario o correo electrónico.
-

Plan de Pruebas Inicial (Aplicación escritorio)

Pruebas de Carga y Disponibilidad Local:

-Verificación de Dependencias Locales: Validar que el entorno de ejecución de Python y las librerías necesarias (Prophet) estén correctamente instaladas en la máquina local y funcionen sin buscar actualizaciones en red.

-Persistencia de Datos: Confirmar que el CRUD (Ingreso de inventario y registro de ventas) guarde la información en la base de datos local incluso tras reinicios forzados del equipo.

Pruebas de Inteligencia Artificial:

-Prueba de Procesamiento Local: Medir el tiempo que tarda el Script de Python en generar la predicción de stock. El resultado debe ser fluido para no entorpecer la gestión del dueño.

-Consistencia de Predicción: Ingresar datos históricos de ventas simulados en la base de datos local y verificar que Prophet genere gráficos de tendencia coherentes en el dashboard del dueño.

Pruebas Funcionales de Seguridad:

-Control de Acceso (RBAC): Verificar que, al no haber validación externa (nube), el sistema local maneje correctamente los roles de Administrador y Vendedor mediante la base de datos encriptada

-Integridad de Auditoría (Logs): Comprobar que cada ajuste manual de stock quede registrado localmente con su "Nota de Motivo", usuario y fecha, garantizando la trazabilidad sin conexión.

Plan de Pruebas Inicial (Interfaz web)

Pruebas de Visualización y Contenido:

-Integridad de la Carta Digital: Verificar que los productos, descripciones y precios mostrados en la web coincidan exactamente con lo ingresado en el sistema de inventario local.

-Carga de Imágenes: Confirmar que las fotos de los productos y del local carguen correctamente y no afecten el rendimiento de la página.

-Enlaces de Redes Sociales: Validar que todos los iconos de redes sociales redirijan correctamente a los perfiles oficiales de la cafetería.

Pruebas de usabilidad:

-Diseño Mobile-First: Comprobar que el menú de navegación sea fácil de operar con una sola mano en dispositivos móviles.

-Jerarquía de Navegación: Asegurar que el usuario no necesite más de 3 clics para encontrar la carta de productos o la información de contacto.

-Formulario de Contacto: Verificar que, al completar el formulario, el sistema genere la notificación correspondiente al administrador (vía local o correo si hay conexión puntual).

Reuniones daily

Fecha	Miembro del Equipo	¿Qué logré ?	¿Qué haré hoy ?	¿Tengo impedimentos?	Notas/Parking Lot
16/03	Isaac, Almendra, Yerk o	planteamiento de proyecto	documentación	no	
19/03	Isaac, Almendra, Yerk o	análisis, diagramas, requisitos	finalizar documentación	no	
15/04	Isaac, Almendra, Yerk o	creación de la arquitectura base del sistema y diseñamos la página principal en conjunto	desarrollo principal de la página web	no	
30/04	Isaac, Almendra, Yerk o	creación del sistema de inventario	arreglar detalles de los paneles	no	
10/05	Isaac, Almendra, Yerk o	implementación ia en el sistema de inventario	probar si la ia funciona correctamente	no	
18/05	Isaac, Almendra, Yerk o	terminamos detalles y la documentación del proyecto	finalizar documentacion y probar el proyecto	no	

Registro de impedimento

ID	Fecha de Reporte	Descripción del Impedimento	Prioridad	Dueño (Asignado a)	Estado	Fecha de Resolución
1	2-5-2026	bug en tabla ventas. cuando se agrega ventas a una tabla vacía no se muestra en pantalla (bug visual)	alta	Yerko	Solucionado	4-5-2026
2	7-5-2026	bug boton vista(no funciona)	media	Isaac	Solucionado	7-5-2026
3	18-5-2026	error en base de datos	muy alta	Yerko	Solucionado	18-5-2026
4	16-5-2026	error matematico calculo costo total produccion platillo	bajo	Isaac- Yerko	Pendiente	
5						

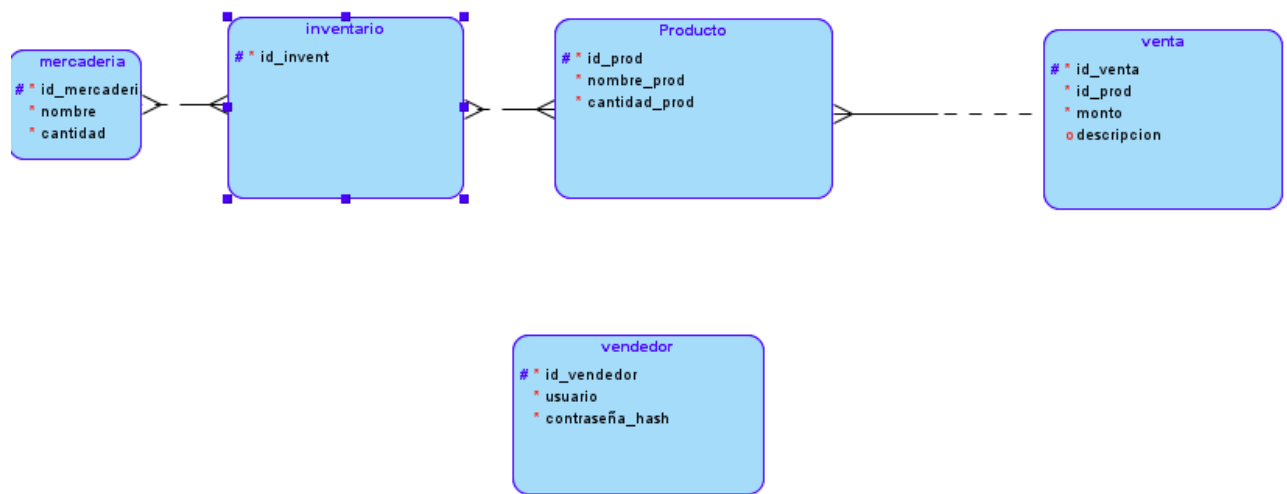
Scrum board

Pendiente	En Progreso	Finalizado

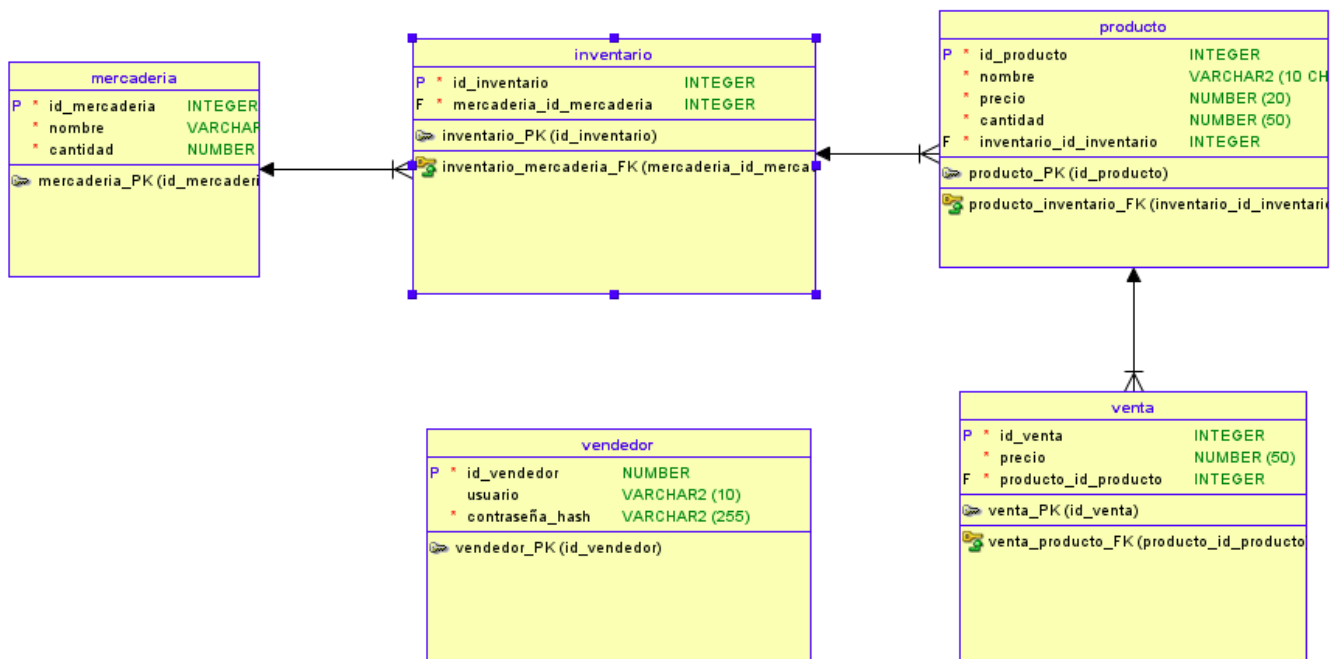
Validar Rol de usuarios	Corregir errores de conexión con Python (IA).	Base de datos SQLite creada y configurada.
Validar formato de RUT en usuarios.	Validar historial mínimo de 7 días para la IA.	Arquitectura base del sistema (Capas completas).
Validar contraseñas seguras en el registro.	Validar stock de ingredientes en recetas de platillos.	Validación de nombre único para productos.
Devolver stock al almacén al anular una venta.	Validar lectura del archivo CSV de datos.	Desactivación lógica de productos vendidos.
Bloquear eliminación de categorías con productos.	Sincronización diaria del equipo (Dailys).	Tabla de auditoría base funcionando.
Bloquear el ingreso de precios o cantidades negativas.		Documentación, matriz y diagramas de secuencia.

<https://trello.com/invite/b/69bc462e990ec469540ec038/ATTle2264dc8ec3cc98704bec03bc962b7bf4E18D23D/proyecto-local-de-comida> / tareas con mayor detalle

Modelo Entidad - Relación



Modelo Relacional



Arquitectura

AppVentasInventario:

El proyecto ocupa dos arquitecturas principales

MVC (Modelo vista controlador): estándar de aplicaciones Java FX.

Arquitectura en capas(N-tier): para la separación de responsabilidades en la capa de negocio(service) y la capa de acceso a datos(repository)

Pagina web:

Component-Based Architecture

Diccionario de datos

1. Módulo de Usuarios

- **crearUsuario(Usuario usuario)**: Registrar nuevo personal.
- **listarUsuarios()**: Mostrar todos los empleados.
- **buscarUsuarioPorRut(String rut)**: Buscar un usuario
- **actualizarUsuario(Usuario usuario)**: Modificar datos existentes.
- **desactivarUsuario(int id)**: Cambio de estado en vez de eliminar.
- **tienePermiso(Usuario usuario, String modulo)**: Acceder a distintos módulos según permiso.
- Usuario iniciarSesion (String rut, String contraseña);
- Usuario eliminarUsuario (String rut)

2. Módulo de Productos (Catálogo)

- **crearProducto(Producto producto)**: Agregar nuevo ítem al menú/inventario.
- **obtenerProductos()**: Ver todo el catálogo disponible.
- **obtenerProductosPorNombre(String nombre)**: Buscador rápido para el vendedor.
- **obtenerProductoPorId(int id)**: Buscador rápido para el vendedor.
- **buscarPorTipo(TipoProducto tipoProducto)**: Filtrar producto según su tipo.
- **actualizarProducto(Producto producto)**: Cambiar precios, nombres o imágenes.
- **eliminarProducto(int id)**: Borrar un producto (siempre que no tenga ventas o platillo asociado).
- **desactivarProducto(int id)**: Desactivar un producto, si esta asociado a ventas o platillo.
- **estaAsociadoVentaOPlatillo(int id)**: Busca si el producto si esta asociado.

- **obtenerStockCritico()**: mostrar el bajo stock de algún producto.
- **existeCategoria(String)**: Verifica que exista una categoría

2.1 Platillo

- **crearPlatillo(Platillo platillo)**: Agregar nuevo ítem al menú/inventario.
- **obtenerPlatillos()**: Ver lista platillos.
- **obtenerPlatillosPorNombre(String nombre)**: Buscador rápido para el vendedor.
-

3. Módulo de Ventas (Transacciones)

- **registrarVenta(Venta venta, List<DetalleVenta> detalleVenta)**: guarda la boleta y sus detalles.
- **listarVentas()**: Historial de transacciones realizadas.
- **buscarVentaPorPeriodo(Fecha inicio, Fecha fin)**: Filtrar ventas para reportes.
- **anularVenta(int idVenta)**: En lugar de eliminar, se marca como "Anulada" por temas contables.

4. Módulo de Inventario (Movimientos y Alertas)

- **registrarMovimiento(Movimiento movimiento)**: Registrar si entró stock (compra) o salió (merma/ajuste).
- **actualizarStockFisico(int idProducto, int cantidad)**: Sumar o restar directamente al `stockActual` del producto.
- **generarAlertaStock()**: Método que busca productos donde `stockActual <= stockMinimo`.
- **listarMovimientosPorProducto(int idProducto)**: Ver el historial de qué ha pasado con un producto específico.
- **ListarMovimientoInventario** (obtener ultimos movimientos)

5. Módulo de Estadísticas e IA (Análisis)

- **obtenerIngresosTotales(periodo)**: Sumatoria de dinero recaudado.
- **obtenerRankingProductos()**: Identificar los productos más vendidos (Top 5).
- **obtenerVentasUsuario(int idUsuario)**: Total de ventas por cada empleado.
- **prepararDatosParaIA()**: Exportar o listar datos históricos de ventas para que el modelo de predicción los procese.

Product Backlog Priorizado:

Seguridad e Infraestructura Base (Prioridad: muy alta)

Sistema de autenticación y RBAC:

-User Story: Como Administrador, quiero que el sistema tenga roles y contraseñas para que un vendedor no pueda ver datos sensibles.

-Criterio de Aceptación: Implementación de Login con cifrado BCrypt y restricción de módulos según rol (Admin/Vendedor).

Configuración de base de datos y respaldos:

-User Story: Como administrador, quiero que el sistema haga respaldos automáticos para no perder información.

-Criterio de Aceptación: Script local que genere una copia de seguridad diaria con capacidad de restauración en menos de 2 horas.

Gestión de Inventario - CRUD (Prioridad: alta)

Módulo de Gestión de Productos:

-User Story: Como Administrador, quiero poder crear, editar y eliminar un producto.

-Criterio de Aceptación: Pantalla funcional de CRUD con validación de borrado lógico para productos con historial

Registro de Entradas y Ajustes Manuales:

-User Story: Como administrador, quiero registrar la entrada de mercadería y realizar ajustes manuales por pérdida dejando una nota.

-Criterio de Aceptación: Formulario de ajuste que obligue a ingresar el motivo y lo guarde en un log de auditoría.

Punto de Venta - POS (prioridad: alta)

Registro de ventas diario:

-User Story: Como vendedor, necesito registrar las ventas del día, agregando pedido, precio y resumen diario.

-Criterio de Aceptación: Interfaz de caja que reste stock automáticamente y genere un resumen de ventas al cierre.

Anulación de Transacciones:

-User Story: Como Administrador, quiero poder anular una venta realizada por error para reintegrar el stock.

-Criterio de Aceptación: Función de anulación que descuente el dinero del total y actualice el inventario en tiempo real

Inteligencia Artificial Predictiva (Prioridad: media)

Dashboard de Análisis y Predicción (Prophet).

-User Story: Como dueño, me gustaría tener un gráfico para análisis de cómo va mi negocio y balance de inventario agotándose.

-Criterio de Aceptación: Integración de script de Python con Prophet para mostrar proyecciones de stock basadas en historial local.

Presencia Digital e Información (Prioridad: baja)

Página Web Informativa y Carta Digital.

-User Story: Como cliente, quiero ver una página intuitiva que funcione en PC y teléfono para ver los productos.

-Criterio de Aceptación: Sitio web 100% responsivo (Mobile-First) con la carta actualizada y botones de redes sociales.

Especificación de la lógica (reglas de negocio)

Módulo: Gestión de Usuarios y Accesos

Objetivo: Administrar quién entra al sistema y qué permisos tiene dentro de la Cafetería Eluney.

RN-USU-01: Autenticación de Usuarios (Login)

- Entrada: rut y Contraseña.
- Lógica de Negocio:
 1. Buscar el usuario en la base de datos por su rut.
 2. Verificación de Estado: Si el usuario existe pero su estado es false (Inactivo), denegar acceso con el mensaje: *"Usuario deshabilitado. Contacte al administrador"*.
 3. Cotejo de Credenciales: Comparar la contraseña ingresada con la almacenada (idealmente usando un hash, no texto plano).
- Salida: Si es correcto, el Service retorna el objeto Usuario completo (incluyendo su Role) para que la UI sepa qué menús mostrar.

RN-USU-02: Validación de Identidad Única

- Evento: Al crear o editar un usuario.
- Lógica: Antes de guardar, el Service debe consultar si ya existe un Rut o un nombre de usuario idéntico.
- Acción: Si el Rut ya existe, se debe lanzar una excepción de negocio que detenga el proceso y avise: *"El RUT ingresado ya pertenece a otro colaborador"*.

RN-USU-03: Control de Roles y Permisos (RBAC)

- Lógica: El Service debe exponer un método llamado tienePermiso(Usuario usuario, String modulo).
- Regla de Negocio:
 - Si Rol == ADMIN, tiene acceso total (Inventario, Usuarios, Reportes, Ventas globales).
 - Si Rol == VENDEDOR, el Service debe bloquear o retornar false si se intenta acceder a "Reportes globales" o "Gestión de Usuarios". Solo acceder a reporte de sus ventas.

RN-USU-04: Eliminación Lógica (Soft Delete)

- Descripción: Nunca borramos un usuario físicamente de la base de datos.
- Por qué: Si borras al usuario "Juan", todas las ventas que hizo Juan en el pasado quedarían sin autor (huérfanas).

- **Lógica:** El método `eliminarUsuario(int id)` del Service solo debe cambiar el atributo estado a false.
- **Efecto:** El usuario ya no podrá loguearse, pero su nombre seguirá apareciendo en los reportes de ventas históricos.

Flujo de Lógica

UsuarioService flujo de registro de usuario.

1. **Recibir datos** del Controller.
2. **Validar formato:** (Que el RUT sea válido, que la clave tenga el largo mínimo).
3. **Validar duplicidad:** (Consultar al Repository si el RUT existe).
4. **Cifrar:** (Pasar la clave por un algoritmo de seguridad).
5. **Persistir:** (Llamar al Repository para insertar en la BD).

Módulo: Gestión de Productos

Objetivo: Administrar la definición, precios y clasificación de todos los ítems de la Cafetería Eluney.

RN-PROD-01: Integridad del Catálogo (Creación/Edición)

- **Descripción:** Asegura que cada producto en el sistema sea único y esté bien categorizado.
- **Lógica de Negocio:**
 1. **Validación de Unicidad:** Antes de guardar, verificar que el nombre o código no exista. El sistema es sensible a duplicados (no queremos dos registros de "Leche Colun").
 2. **Clasificación Obligatoria:** Todo producto debe ser marcado como:
 - **Producto Directo:** Se compra y vende tal cual (ej. Lata de Bebida).
 - **Platillo:** Requiere una receta para existir (ej. Torta).
 - **Solo inventario:** Insumo solo para inventario
- **Acción:** Si falta una categoría o el nombre existe, el Service lanza una excepción.

RN-PROD-02: Política de Precios y Márgenes

- **Descripción:** Protege la rentabilidad del negocio.
- **Lógica:**
 1. **Validación de Margen:** El precio de Venta debe ser obligatoriamente mayor al precio de Compra (para productos directos).

2. **Costo de Platillo:** Si el producto es un **Platillo**, el precio de Compra (costo) no se ingresa manualmente; el Service debe calcularlo sumando los costos de sus ingredientes (procedentes del PlatilloService).
- **Acción:** Bloquear el guardado si el margen es menor al 10% (opcional, según política de la cafetería) o si es negativo.

RN-PROD-03: Gestión de Unidades de Medida

- **Descripción:** Define cómo se cuantifica el producto en todo el sistema.
- **Lógica:** * El Service debe validar que la unidad elegida coincida con el tipo de producto.
 - **Regla:** Los productos pesables (café, harina) deben usar unidades de masa/volumen. Los productos unitarios (bebidas, postres terminados) deben usar "Unidad".
- **Importante:** Una vez que un producto tiene movimientos de inventario, el ProductoService **no debe permitir cambiar la unidad de medida** para evitar descuadres históricos.

RN-PROD-04: Estado y Visibilidad (Eliminación Lógica)

- **Descripción:** Controla si un producto está disponible para la venta o uso en recetas.
- **Lógica:**
 1. **Verificación de Dependencia:** Antes de desactivar un producto, el Service consulta: ¿Es este producto ingrediente de algún platillo activo?
 2. **Acción:** Si hay dependencia, denegar la desactivación. Si no la hay, cambiar estado a desactivado y el producto no sigue disponible para uso.
 3. **No Borrado Físico:** Nunca eliminar directamente. Solo cambiar el estado para mantener el historial de ventas antiguas.

flujo

Entrada: Recibe el objeto del controlador.

Validación: Aplica las reglas (duplicidad, márgenes, dependencias).

Ejecución: Envía al DAO/Repository para guardar.

Respuesta: Retorna un mensaje de éxito o lanza una excepción clara.

Módulo: Gestión de Ventas (Transacciones)

Objetivo: Registrar, controlar y asegurar la integridad de cada transacción comercial en la Cafetería Eluney, vinculando la salida de productos con el ingreso financiero.

RN-VENT-01: Garantía de Disponibilidad (Validación de Stock)

Descripción: Evita que se realicen ventas de productos que no están físicamente en estantería. **Lógica de Negocio:**

- **Verificación Previa:** Por cada ítem en el `List<DetalleVenta>`, el Service debe consultar al `ProductoService` el stock actual.
- **Regla de Negocio:** Si `cantidad_venta > stock_actual`, la transacción debe ser rechazada inmediatamente.
- **Excepción:** No se permiten ventas con "stock negativo". Si un producto es un **Platillo**, la validación se hace sobre el producto terminado, no sobre sus insumos (estos ya debieron descontarse al producir el platillo).
- **Acción:** Lanzar una excepción detallada indicando qué producto falló y cuánto stock falta.

RN-VENT-02: Integridad Contable (Transaccionalidad)

Descripción: Asegura que la boleta y sus detalles se guarden como una única unidad de información. **Lógica de Negocio:**

- **Todo o Nada:** Si el guardado de un solo `DetalleVenta` falla, la cabecera de la `Venta` no debe persistir en la base de datos.
- **Sincronización de Inventario:** El Service debe orquestar que, tras un guardado exitoso en el Repository, se llame automáticamente al método de actualización de stock para cada producto.
- **Acción:** Ejecutar el registro dentro de una transacción de base de datos (manejada por el DAO/Repository bajo orden del Service).

RN-VENT-03: Política de Anulación (No Eliminación)

Descripción: Regula cómo deshacer una venta sin perder el rastro de auditoría. **Lógica de Negocio:**

- **Estado de Transacción:** Las ventas nunca se borran de la base de datos. El Service cambia su estado a "ANULADA".
- **Reversa de Inventario:** Al anular, el Service debe realizar la operación matemática inversa: devolver las cantidades de los detalles al `stock_actual` de cada producto.
- **Restricción Temporal:** (Opcional) El Service puede denegar anulaciones de ventas que tengan más de X días de antigüedad para proteger la contabilidad mensual.
- **Acción:** Actualizar estado en `Venta` y sumar cantidades en `Producto`.

RN-VENT-04: Consistencia en Reportes (Filtrado de Periodos)

Descripción: Asegura que los datos entregados para estadísticas e IA sean veraces. **Lógica de Negocio:**

- **Validación Cronológica:** El Service debe impedir que la `fecha_inicio` sea posterior a la `fecha_fin`.

- **Exclusión de Ruido:** Los reportes para la IA (Prophet) deben filtrar únicamente ventas con estado "COMPLETADA". Las ventas anuladas se ignoran para no inflar artificialmente la predicción de demanda.
-

Flujo de Lógica en VentaService

1. **Entrada:** Recibe el objeto **Venta** y una **List<DetalleVenta>** desde el controlador.
2. **Validación:** * Verifica stock de cada producto.
 - Calcula que el **total_venta** coincida con la suma de los detalles (validación de integridad aritmética).
3. **Ejecución:** * Solicita al Repository guardar la transacción.
 - Si tiene éxito, solicita al **ProductoService** descontar el inventario.
4. **Respuesta:** Retorna el ID de la venta generada o lanza una excepción con el motivo del fallo (ej. "Stock Insuficiente" o "Error de Conexión").

Módulo: Control de Inventario y Movimientos

Objetivo: Gestionar el flujo físico de mercancías, garantizar la trazabilidad de cada gramo/unidad y disparar alertas preventivas.

RN-INV-01: Validación de Disponibilidad (El "Pre-Check")

- **Descripción:** Antes de que cualquier otro módulo (Ventas o platillo) intente restar stock, este método debe dar "luz verde".
- **Lógica de Negocio:**
 1. Recibe una lista de **ID_Producto** y **Cantidad_Requerida**.
 2. Por cada ítem, verifica: **Stock_Actual >= Cantidad_Requerida**.
 3. **Importante:** Si el ítem es un **Platillo**, el servicio debe llamar internamente a la lógica de "Explosión de Receta" para verificar si hay suficientes insumos (café, leche, etc.) para completar ese platillo.
- **Salida:** Retorna un Booleano o una lista de productos faltantes.

RN-INV-02: Registro Obligatorio de Movimientos

- **Descripción:** No puede existir un cambio en el stock sin un registro que explique el "por qué".
- **Lógica:** Cada vez que se ejecute un método de actualización de stock, el Service debe insertar una fila en la tabla **MovimientoInventario** con:

- **TipoMovimiento:** (ENTRADA / SALIDA / MERMA / AJUSTE).
- **Motivo:** (VENTA / COMPRA / MERMA / VENCIMIENTO).
- **UsuarioID:** Quién autorizó el movimiento.
- **Cantidad:** El valor del cambio.

RN-INV-03: Lógica de Alertas de Stock Crítico

- **Descripción:** Automatiza la vigilancia de las existencias.
- **Lógica:** 1. Después de cada operación de **Salida**, el Service compara: `Stock_Actual <= Stock_Minimo`. 2. Si se cumple, el Service invoca al `AlertaService.crearAlerta(producto)`. 3. Si ocurre una **Entrada** y el stock supera el mínimo, el Service debe buscar si hay una alerta activa para ese producto y marcarla como "RESUELTA".

RN-INV-04: Gestión de Mermas y Desperdicios

- **Descripción:** Controla las pérdidas no relacionadas con ventas (ej. leche vencida, taza rota).
- **Lógica:**
 1. Requiere un **Motivo** detallado y la **Cantidad** a restar.
 2. Calcula el "Costo de la Merma": `Cantidad * Precio_Costo`. Esto es vital para los reportes de pérdida económica a fin de mes.

Flujo de Lógica Técnica

Cuando el sistema procesa, por ejemplo, una **Entrada por Compra**, el `InventarioService` sigue estos pasos:

1. **Recibir:** ID Producto + Cantidad.
2. **Transacción:** Abrir transacción de Base de Datos.
3. **Actualizar:** `UPDATE Productos SET stock_actual = stock_actual + cantidad WHERE id = ?`.
4. **Registrar:** `INSERT INTO Movimientos (...)`.
5. **Limpiar:** Si `stock_actual > stock_minimo`, desactivar banderas de alerta.
6. **Commit:** Guardar cambios permanentemente.

Módulo: Inteligencia de Negocio y Estadísticas

Objetivo: Consolidar los datos de ventas e inventario para generar reportes históricos y alimentar el script de IA Predictiva (Prophet).

RN-EST-01: Cálculo de Rentabilidad (Ventas vs. Costos)

- **Descripción:** Determinar cuánto dinero real está ganando la cafetería tras descontar el costo de los insumos.
- **Lógica de Negocio:**
 1. Recuperar las ventas de un periodo (Día/Mes).
 2. Por cada `DetalleVenta`, restar el `precioCosto` (que viene de la receta o del producto directo) del `precioVenta`.
 3. **Fórmula:** `Utilidad = Suma(Ventas) - Suma(Costos de Insumos)`.
- **Salida:** Un valor decimal que representa la ganancia neta.

RN-EST-02: Identificación de mejores Productos (Ranking)

- **Descripción:** Listar los productos y platillos con mayor demanda.
- **Lógica:**
 1. Agrupar los `DetalleVenta` por `ID_Producto`.
 2. Sumar las cantidades vendidas en el rango de fechas seleccionado.
 3. Ordenar de mayor a menor.
- **Visualización:** Gráfico de barras que muestra el "Top 5" de productos.

RN-EST-03: Preparación de Datos para IA (Exportación a Python)

- **Descripción:** Este es el puente con el script de **Prophet**.
- **Lógica:**
 1. El Service debe generar un dataset (preferiblemente en formato CSV o JSON).
 2. **Estructura requerida por Prophet:** Necesitas al menos dos columnas: `ds` (datestamp/fecha) y `y` (valor a predecir, ej. ventas por día).
 3. El Service limpia los datos (elimina ventas anuladas) para no ensuciar la predicción.

RN-EST-04: Análisis de Quiebres de Stock (Stock-out Analysis)

- **Descripción:** Reportar cuántas veces un producto llegó a cero o activó una alerta.
 - **Lógica:**
 1. Contar los registros en la tabla `MovimientoInventario` que dispararon una `AlertaStock`.
 2. Identificar patrones: "*¿Se agota el café siempre los viernes?*".
 - **Propósito:** Ayudar al dueño a ajustar su `StockMinimo` de forma manual o sugerida.
-

Flujo de Lógica en el Módulo de Estadísticas

A diferencia de Ventas o Inventario, este servicio es de **Solo Lectura (Read-Only)** pero con alta carga de procesamiento:

1. **Recibir Filtros:** (Fecha Inicio, Fecha Fin, Categoría).
2. **Consulta (Query):** El Service pide al Repository que haga "Joins" entre Ventas, Detalles y Productos.
3. **Procesamiento:** Se realizan los cálculos matemáticos (promedios, sumas, porcentajes).

Conexión con la IA Predictiva (Prophet)

Dado que mencionaste en tu documentación la integración con Python:

- El `EstadisticaService` será el encargado de ejecutar el comando de consola para correr el script de Python, pasándole los datos históricos.
- Una vez que Python devuelve la predicción, el `EstadisticaService` la lee y la proyecta en el Dashboard de la App de Escritorio.

MATRIZ DE VALIDACIONES

Documento de referencia para registrar y auditar las validaciones presentes en la interfaz de usuario (Capa de Controladores) y en la capa de negocio (Capa de Servicios) del sistema.

Leyenda de Estados

- **Presente:** Validación completamente implementada y operativa.
- **Faltante:** Validación inexistente que requiere desarrollo.
- **Incompleta:** Validación con desarrollo iniciado pero no finalizado.

- **Parcial:** Validación implementada en una sola capa (UI o Service), sin cobertura en la otra.

1. MÓDULO: PRODUCTOS

1.1 Registrar Producto

Validación	Descripción	UI (Controlleur)	Service	Estado	Observaciones
Producto nulo	Validar que el objeto no sea null	Faltante	Presente	Incompleta	Solo en Service
Nombre obligatorio	Nombre no vacío ni nulo	Faltante	Presente	Incompleta	Solo en Service
Nombre único	El nombre no debe repetirse	Presente	Presente	Presente	Validado en ambas capas
Precio compra > 0	El precio de compra debe ser mayor a cero	Faltante	Presente	Incompleta	Solo en Service
Unidad de medida	Debe ser asignada	Faltante	Presente	Incompleta	Solo en Service
Margen mínimo 10%	Precio venta >= (Precio compra x 1.10)	Faltante	Presente	Incompleta	Solo en Service; excepto SOLO_INVENTARIO

Tipo producto SOLO_INVENTARIO	Fuerza precio venta a 0	Faltante	Presente	Incompleta	Solo en Service (lógica automática)
Cantidad obligatoria	Si unidad es GRAMOS/MILILITROS, cantidad > 0	Presente	Faltante	Parcial	Solo en UI (validación visual)
Stock actual >= 0	No permitir stock negativo	Faltante	Faltante	Faltante	Requerido
Stock mínimo >= 0	Stock mínimo no puede ser negativo	Faltante	Faltante	Faltante	Requerido
Auditoría registrada	Se debe registrar el evento en la tabla auditoria	Faltante	Presente	Incompleta	Solo en Service

1.2 Actualizar Producto

Validación	Descripción	UI (Controller)	Service	Estado	Observaciones
Producto existe	Verificar que el ID corresponda a un producto existente	Faltante	Presente	Incompleta	Solo en Service

Nombre obligatorio	Nombre no vacío ni nulo	Faltante	Presente	Incompleta	Solo en Service
Nombre único (excepto el propio)	No duplicar nombre con otros productos	Presente	Faltante	Parcial	Solo en UI (verifica idAExclur)
Precio compra > 0	Precio de compra debe ser positivo	Faltante	Presente	Incompleta	Solo en Service
Unidad de medida	Debe ser asignada	Faltante	Presente	Incompleta	Solo en Service
Auditoría registrada	Se debe registrar el evento de actualización	Faltante	Presente	Incompleta	Solo en Service

1.3 Eliminar Producto

Validación	Descripción	UI (Controller)	Service	Estado	Observaciones
ID válido (> 0)	Producto seleccionado debe tener ID > 0	Faltante	Presente	Incompleta	Solo en Service

Producto existe	Verificar que el producto exista antes de eliminarlo	Faltante	Presente	Incompleta	Solo en Service
Verificar asociación a venta/platillo	Si está asociado, desactivar; si no, eliminar	Faltante	Presente	Presente	Solo en Service (lógica completa)
Auditoría registrada	Registrar evento de eliminación	Faltante	Presente	Incompleta	Solo en Service

2. MÓDULO: PLATILLOS

2.1 Registrar Platillo

Validación	Descripción	UI (Controller)	Service	Estado	Observaciones
Platillo nulo	Objeto platillo no puede ser null	Faltante	Presente	Incompleta	Solo en Service
Nombre obligatorio	Nombre no vacío ni nulo	Faltante	Presente	Incompleta	Solo en Service
Nombre único	El nombre del platillo no debe repetirse	Faltante	Presente	Incompleta	Solo en Service

Precio > 0	Precio de venta debe ser mayor a cero	Faltante	Presente	Incompleta	Solo en Service
Tipo producto = PLATILLO	Solo se pueden registrar platillos con tipo PLATILLO	Faltante	Presente	Incompleta	Solo en Service
Ingredientes no vacíos	Debe tener al menos un ingrediente	Faltante	Faltante	Faltante	Requerido
Ingredientes válidos	Ningún ingrediente puede ser null	Faltante	Presente	Incompleta	Solo en Service
Cantidad ingrediente > 0	Cantidad de ingrediente debe ser positiva	Faltante	Presente	Incompleta	Solo en Service
Auditoría registrada	Se debe registrar el evento en la tabla auditoria	Faltante	Presente	Incompleta	Solo en Service

2.2 Actualizar Platillo

Validación	Descripción	UI (Controller)	Service	Estado	Observaciones

Platillo existe	Verificar que el ID corresponda a un platillo existente	Faltante	Incompleta	Incompleta	Implementación incompleta
Nombre único (excepto propio)	No duplicar nombre con otros platillos	Faltante	Incompleta	Incompleta	Implementación incompleta
Auditoría registrada	Se debe registrar el evento de actualización	Faltante	Incompleta	Incompleta	Implementación incompleta

2.3 Eliminar Platillo

Validación	Descripción	UI (Controller)	Service	Estado	Observaciones
ID válido	Platillo seleccionado debe tener ID > 0	Faltante	Incompleta	Incompleta	Implementación incompleta
Platillo existe	Verificar que el platillo exista	Faltante	Incompleta	Incompleta	Implementación incompleta
Verificar asociación a venta	Si está asociado, desactivar; si no, eliminar	Faltante	Incompleta	Incompleta	Implementación incompleta

Validación	Descripción	UI (Controller)	Service	Estado	Observaciones
Producto nulo	Validar que el objeto no sea null	Faltante	Presente	Incompleta	Solo en Service
Nombre obligatorio	Nombre no vacío ni nulo	Faltante	Presente	Incompleta	Solo en Service
Nombre único	El nombre no debe repetirse	Presente	Presente	Presente	Validado en ambas capas
Precio compra > 0	El precio de compra debe ser mayor a cero	Faltante	Presente	Incompleta	Solo en Service
Unidad de medida	Debe ser asignada	Faltante	Presente	Incompleta	Solo en Service
Margen mínimo 10%	Precio venta >= (Precio compra x 1.10)	Faltante	Presente	Incompleta	Solo en Service; excepto SOLO_INVENTARIO
Tipo producto SOLO_INVENTARIO	Fuerza precio venta a 0	Faltante	Presente	Incompleta	Solo en Service (lógica automática)
Cantidad obligatoria	Si unidad es GRAMOS/MILILITROS, cantidad > 0	Presente	Faltante	Parcial	Solo en UI (validación visual)

Stock actual >= 0	No permitir stock negativo	Faltante	Faltante	Faltante	Requerido
Stock mínimo >= 0	Stock mínimo no puede ser negativo	Faltante	Faltante	Faltante	Requerido
Auditoría registrada	Se debe registrar el evento en la tabla auditoria	Faltante	Presente	Incompleta	Solo en Service

3. MÓDULO: CATEGORÍAS

3.1 Registrar Categoría

Validación	Descripción	UI (Controller)	Service	Estado	Observaciones
Categoría nula	Objeto categoría no puede ser null	Faltante	Incompleta	Incompleta	Implementación deficiente
Nombre obligatorio	Nombre no vacío ni nulo	Faltante	Incompleta	Incompleta	Implementación deficiente
Nombre único	El nombre de la categoría no debe repetirse	Faltante	Incompleta	Incompleta	Implementación deficiente

Auditoría registrada	Se debe registrar el evento	Faltante	Presente	Incompleta	Solo en Service
----------------------	-----------------------------	----------	----------	------------	-----------------

3.2 Actualizar Categoría

Validación	Descripción	UI (Controller)	Service	Estado	Observaciones
Categoría existe	Verificar que la categoría exista	Faltante	Incompleta	Incompleta	Implementación incompleta
Nombre único (excepto propio)	No duplicar nombre	Faltante	Incompleta	Incompleta	Implementación incompleta
Auditoría registrada	Registrar evento de actualización	Faltante	Presente	Incompleta	Solo en Service

3.3 Eliminar Categoría

Validación	Descripción	UI (Controller)	Service	Estado	Observaciones
------------	-------------	-----------------	---------	--------	---------------

ID válido	Categoría seleccionada debe tener ID > 0	Faltante	Incompleta	Incompleta	Implementación incompleta
Categoría existe	Verificar que la categoría exista	Faltante	Incompleta	Incompleta	Implementación incompleta
No asociada a productos	No permitir eliminar si tiene productos asociados	Faltante	Incompleta	Incompleta	Implementación incompleta
Auditoría registrada	Registrar evento de eliminación	Faltante	Presente	Incompleta	Solo en Service

4. MÓDULO: USUARIOS

4.1 Registrar Usuario

Validación	Descripción	UI (Controller)	Service	Estado	Observaciones
Usuario nulo	Objeto usuario no puede ser null	Faltante	Presente	Incompleta	Solo en Service
Nombre obligatorio	Nombre no vacío ni nulo	Faltante	Presente	Incompleta	Solo en Service

Apellido obligatorio	Apellido no vacío ni nulo	Faltante	Presente	Incompleta	Solo en Service
RUT obligatorio	RUT no vacío ni nulo	Faltante	Presente	Incompleta	Solo en Service
RUT válido	Formato de RUT correcto (XX.XXX.XXX-X)	Faltante	Faltante	Faltante	Requerido
Contraseña obligatoria	Contraseña no vacía ni nula	Faltante	Presente	Incompleta	Solo en Service
Contraseña fuerte	Mínimo 8 caracteres, mayúscula, número y símbolo	Faltante	Faltante	Faltante	Requerido
RUT único	No se pueden registrar dos usuarios con el mismo RUT	Faltante	Presente	Incompleta	Solo en Service

4.2 Iniciar Sesión

Validación	Descripción	UI (Controller)	Service	Estado	Observaciones
Campos no vacíos	RUT y contraseña no deben estar vacíos	Presente	Presente	Presente	Validado en ambas capas

Usuario existe	Verificar que el usuario exista con ese RUT	Faltante	Presente	Incompleta	Implícito en repository
Contraseña correcta	Validar que la contraseña coincida	Faltante	Presente	Incompleta	Implícito en repository
Usuario activo	Solo usuarios con estado = 1 pueden iniciar sesión	Faltante	Faltante	Faltante	Requerido

4.3 Eliminar Usuario

Validación	Descripción	UI (Controller)	Service	Estado	Observaciones
RUT no vacío	RUT debe ser proporcionado	Faltante	Presente	Incompleta	Solo en Service
Usuario existe	Verificar que el usuario exista antes de eliminar	Faltante	Presente	Incompleta	Solo en Service

Usuario no es admin	No permitir eliminar el usuario administrador	Faltante	Faltante	Faltante	Requerido
---------------------	---	----------	----------	----------	-----------

6. MÓDULO: INVENTARIO

6.1 Disponibilidad de Stock

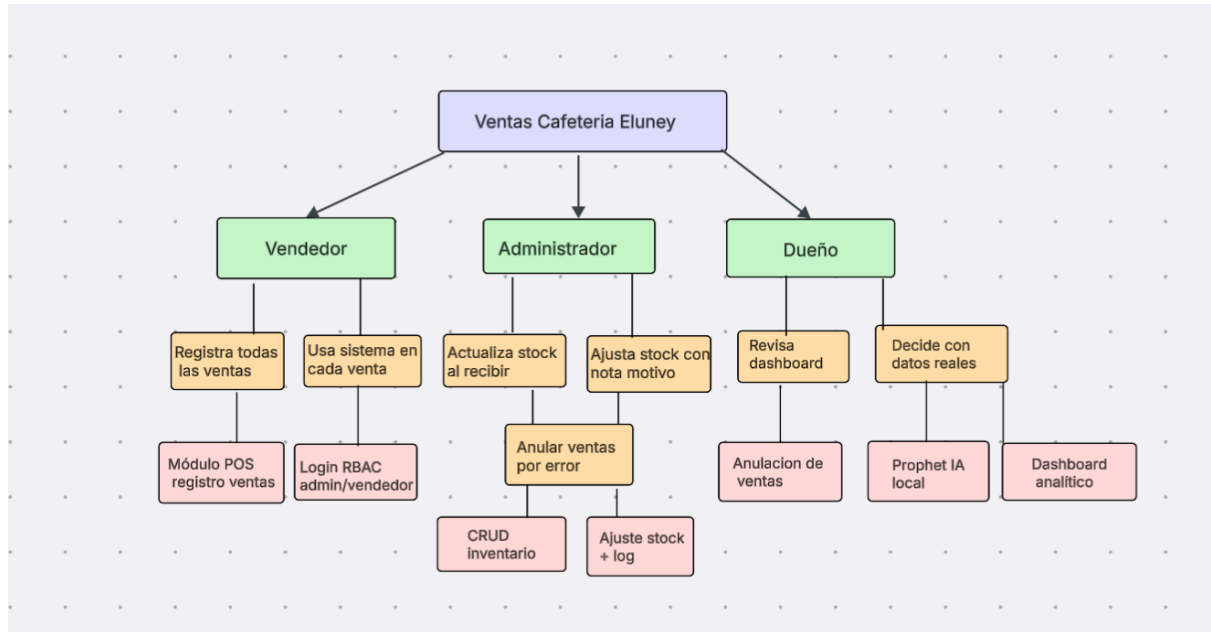
Validación	Descripción	UI (Controller)	Service	Estado	Observaciones
ID producto válido	ID debe ser > 0	Faltante	Presente	Incompleta	Solo en Service
Cantidad requerida > 0	Cantidad debe ser positiva	Faltante	Faltante	Faltante	Requerido
Stock suficiente	Stock >= cantidad requerida	Faltante	Presente	Incompleta	Método validarDisponibilidad
Explosión de receta	Para platillos, validar la disponibilidad de cada ingrediente	Faltante	Incompleta	Incompleta	Pendiente de resolver comentario técnico

7. MÓDULO: ESTADÍSTICAS E IA PREDICTIVA

7.1 Predicción de Stock

Validación	Descripción	UI (Controller)	Service	Estado	Observaciones
Datos suficientes	Mínimo de 7 registros en el historial de ventas	Faltante	Incompleta	Incompleta	Documentado en diagramas, pendiente en código
Productos existen	Todos los identificadores deben ser válidos	Faltante	Incompleta	Incompleta	Implementación incompleta
CSV válido	El archivo de intercambio generado debe ser legible	Faltante	Incompleta	Incompleta	Implementación incompleta
Python disponible	El script Prophet debe ejecutarse sin errores de entorno	Faltante	Incompleta	Incompleta	Incidentes de integración reportados

Impact Mapping:



Casos de uso específicos (UML)

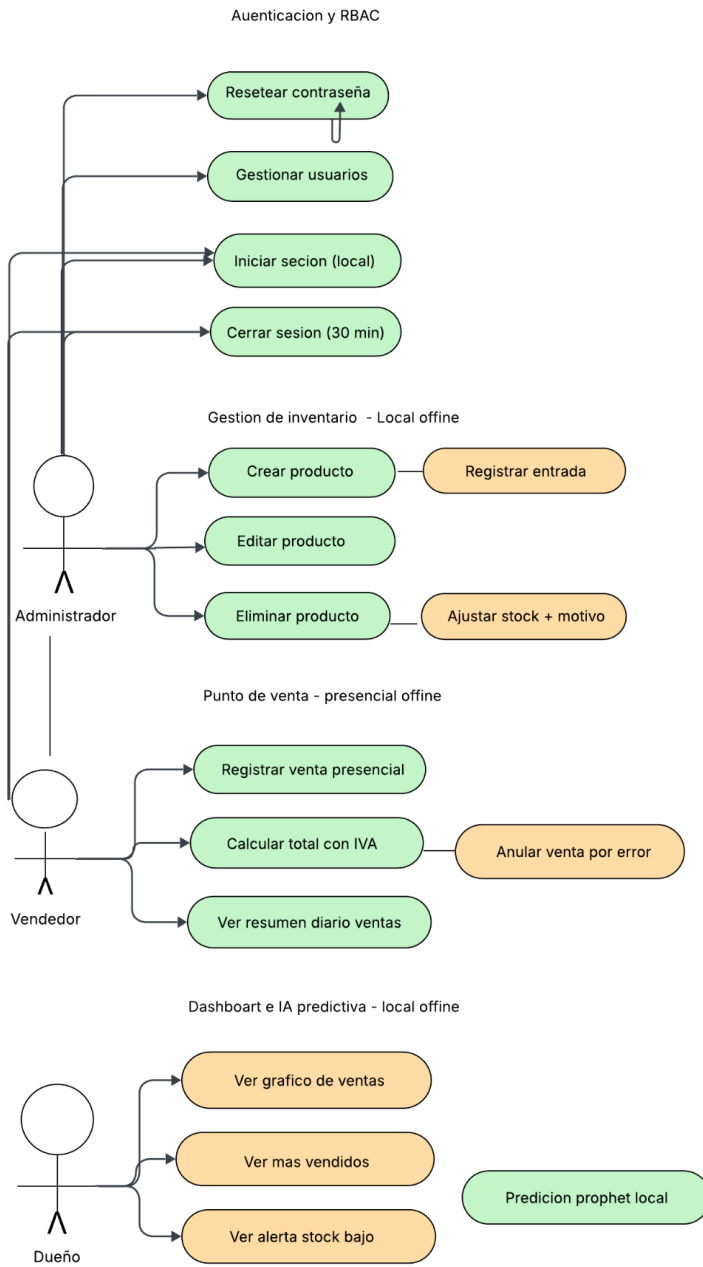
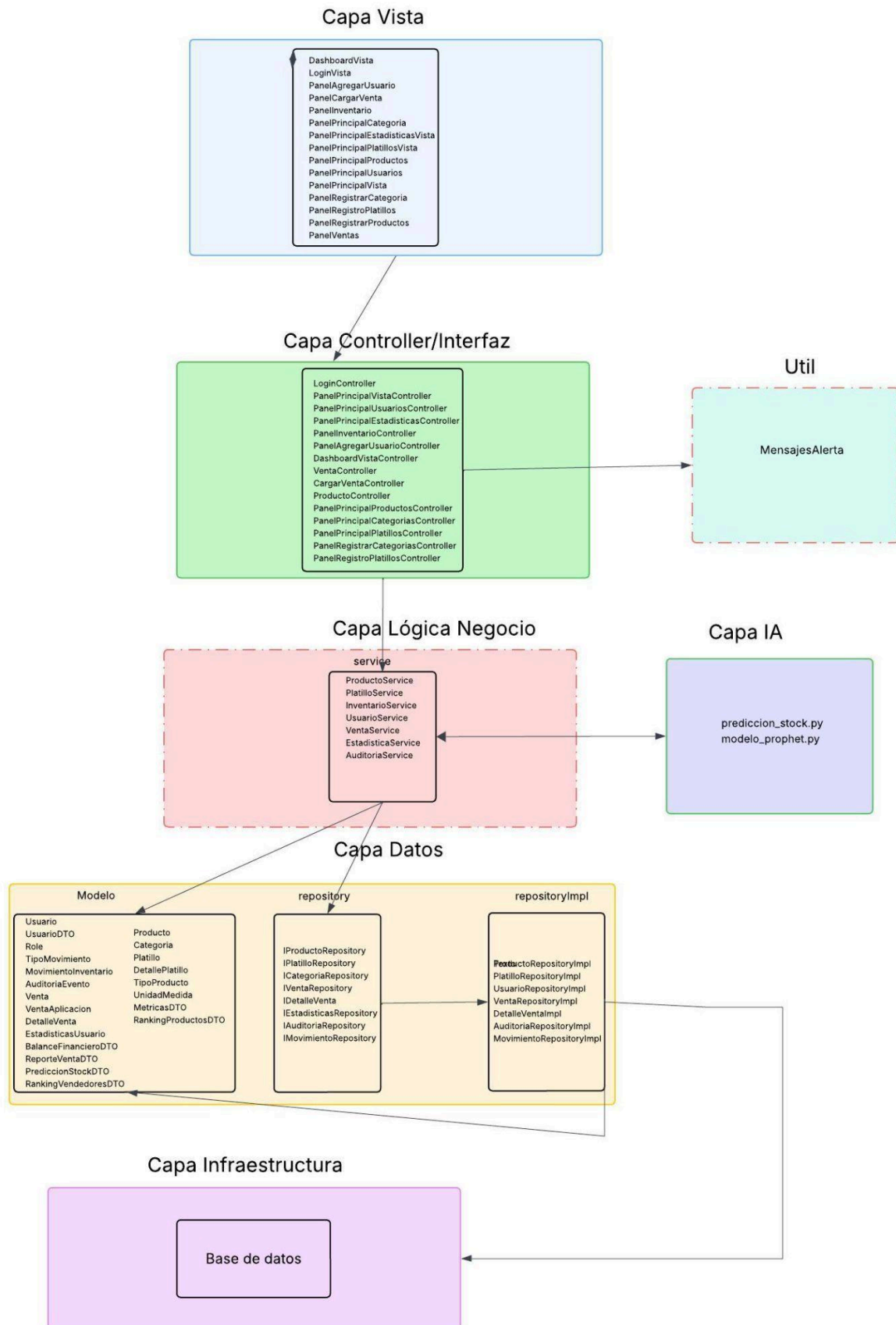
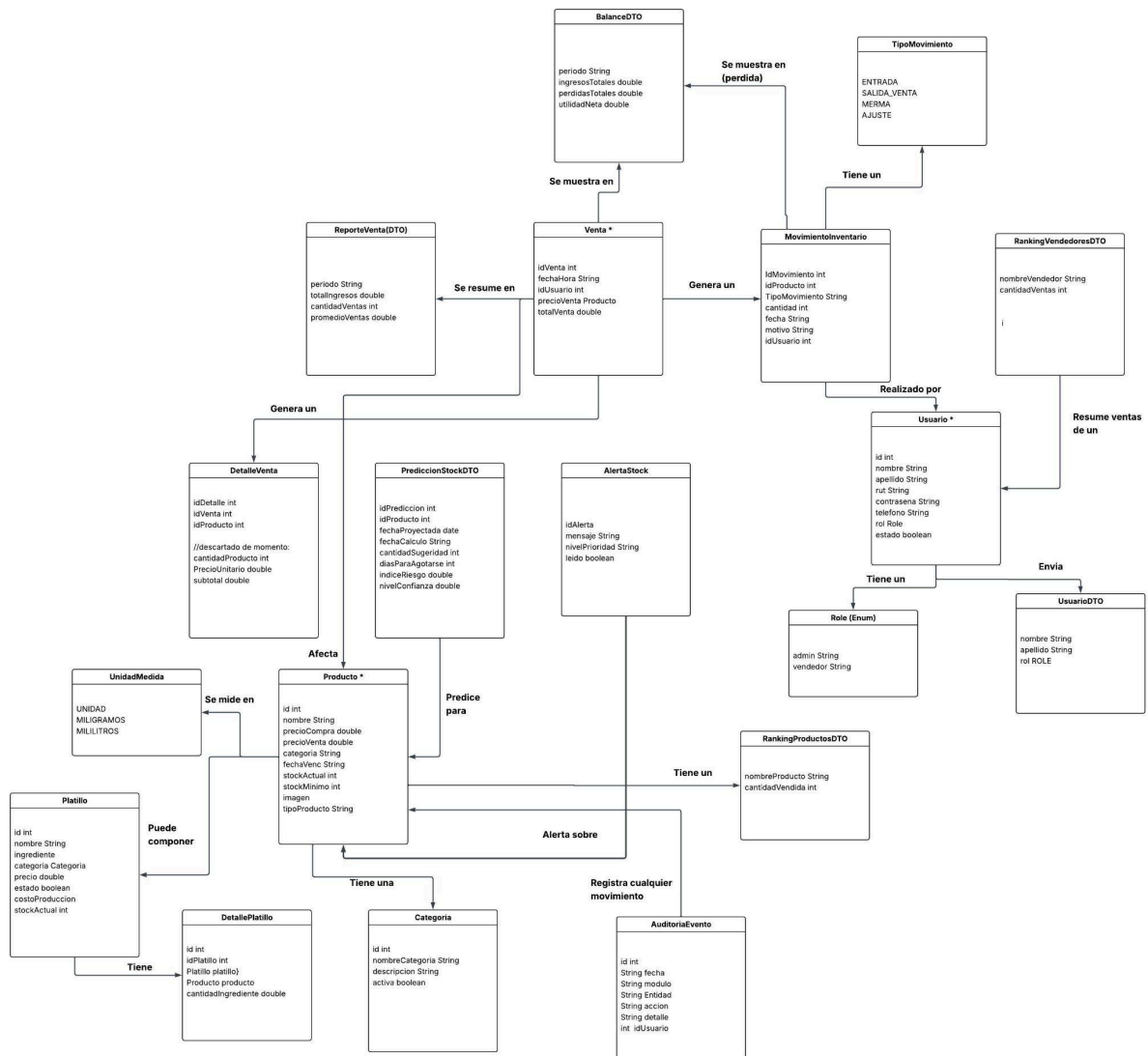


Diagrama de paquetes



https://lucid.app/lucidchart/d68fdb77-7876-422a-8731-5fbdb5c13b3a/edit?viewport_loc=-924%2C84%2C2040%2C1011%2C0_0&invitationId=inv_257b0fce-a7f6-4a0e-8811-f5a3fb5a4dff para ver mejor

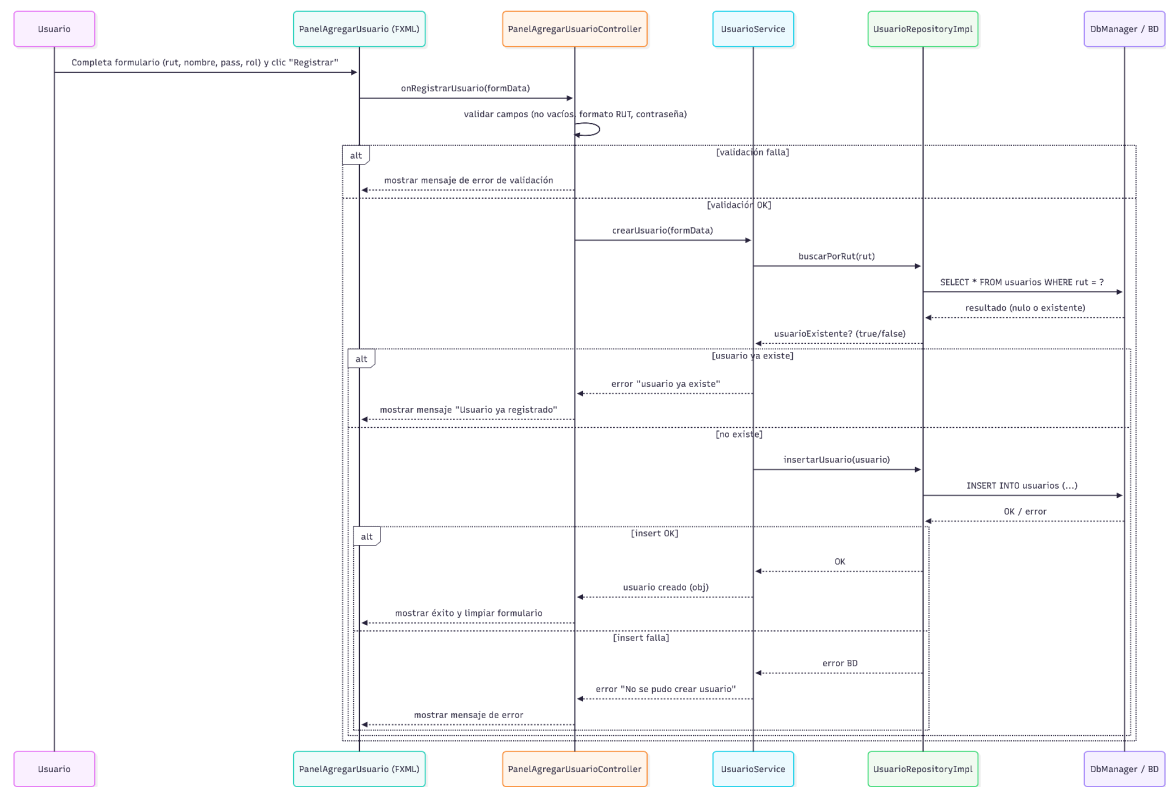
Diagrama de clases



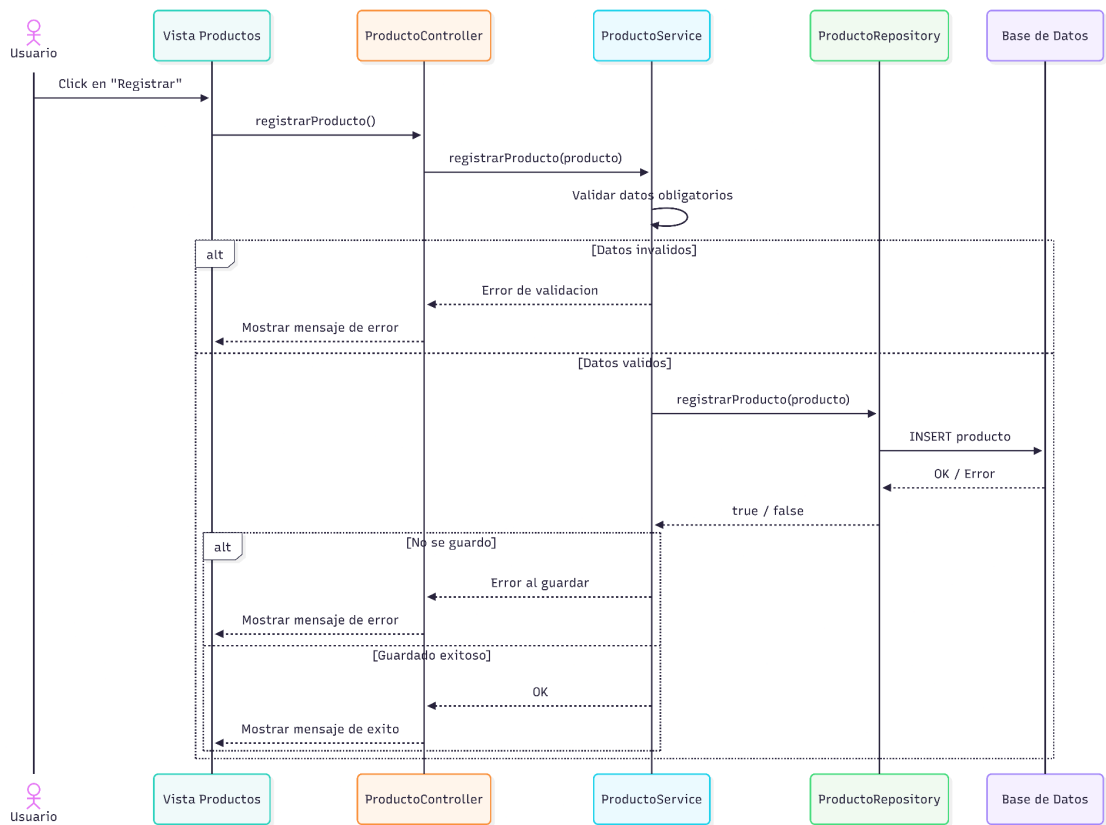
<https://docs.google.com/document/d/159dgRVANQtanFbsdXQtdYEMTfUJcEJyfJHmIFUHVEwk/edit?usp=sharing> / enlace para revisar

Diagrama de secuencia

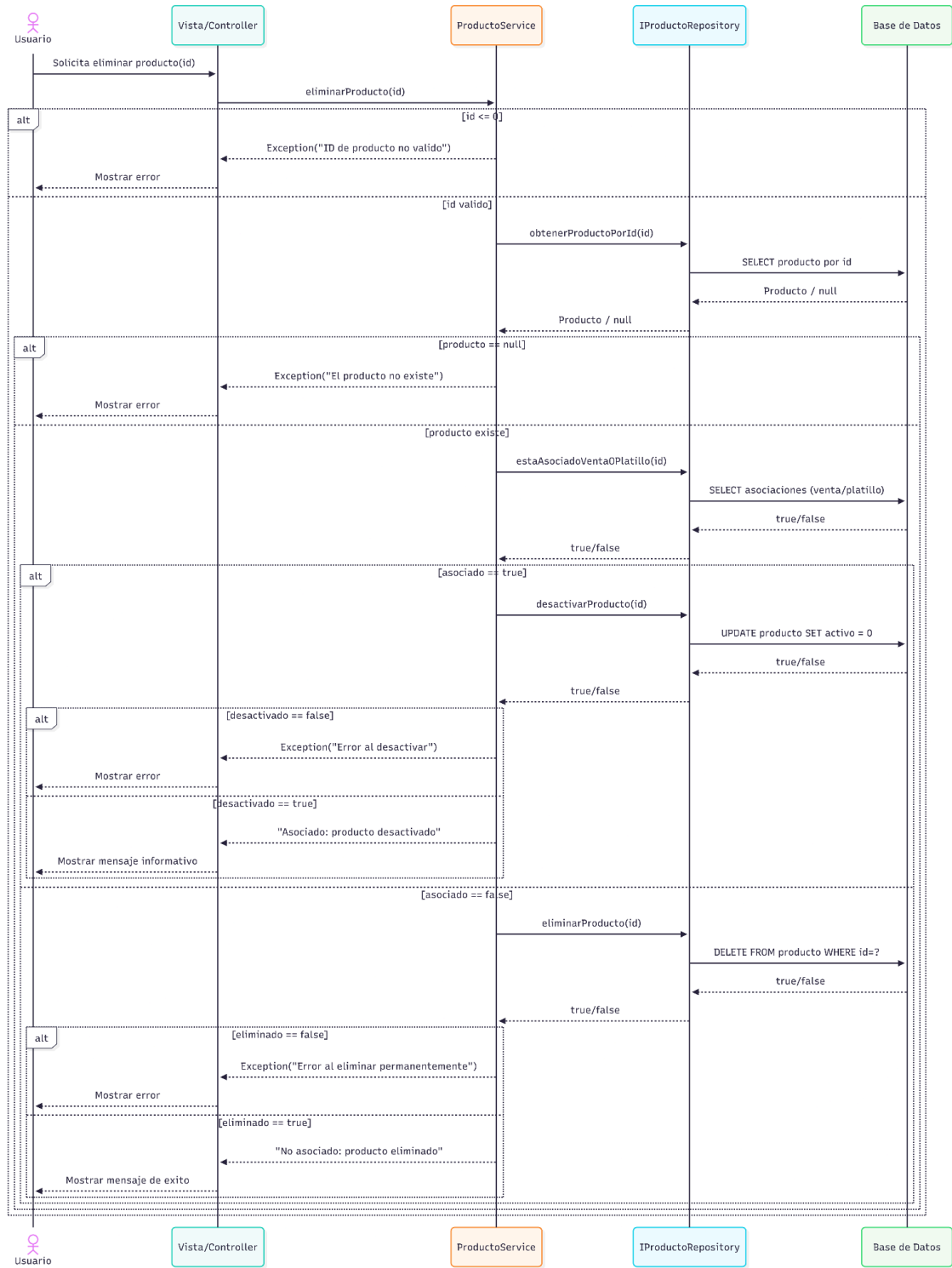
flujo registrar un usuario:



Flujo registrar Producto

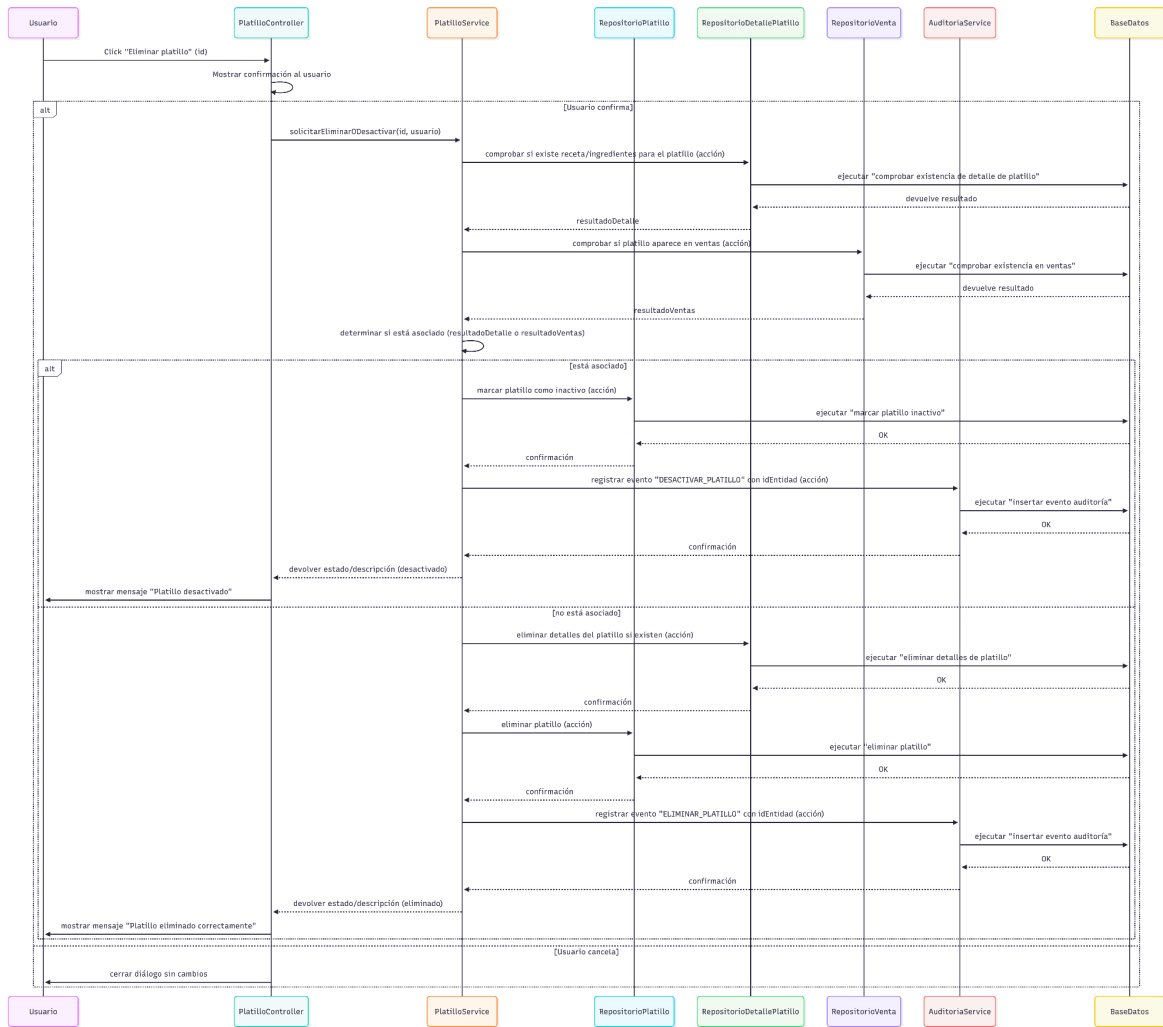


Flujo Eliminar/desactivar Producto

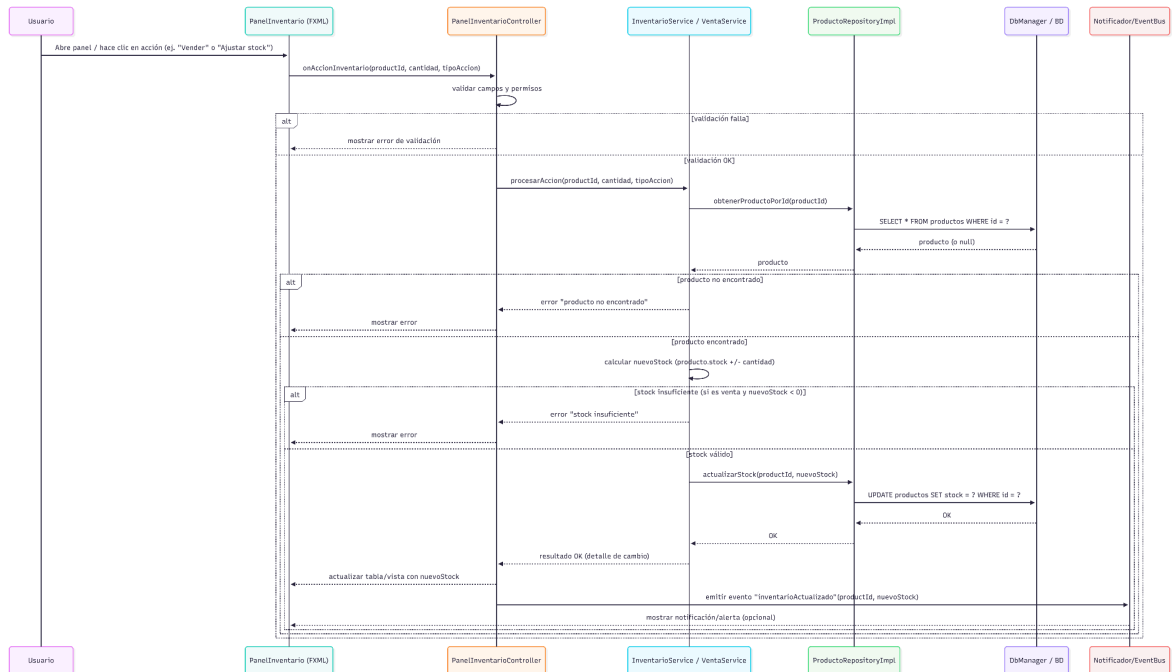


}

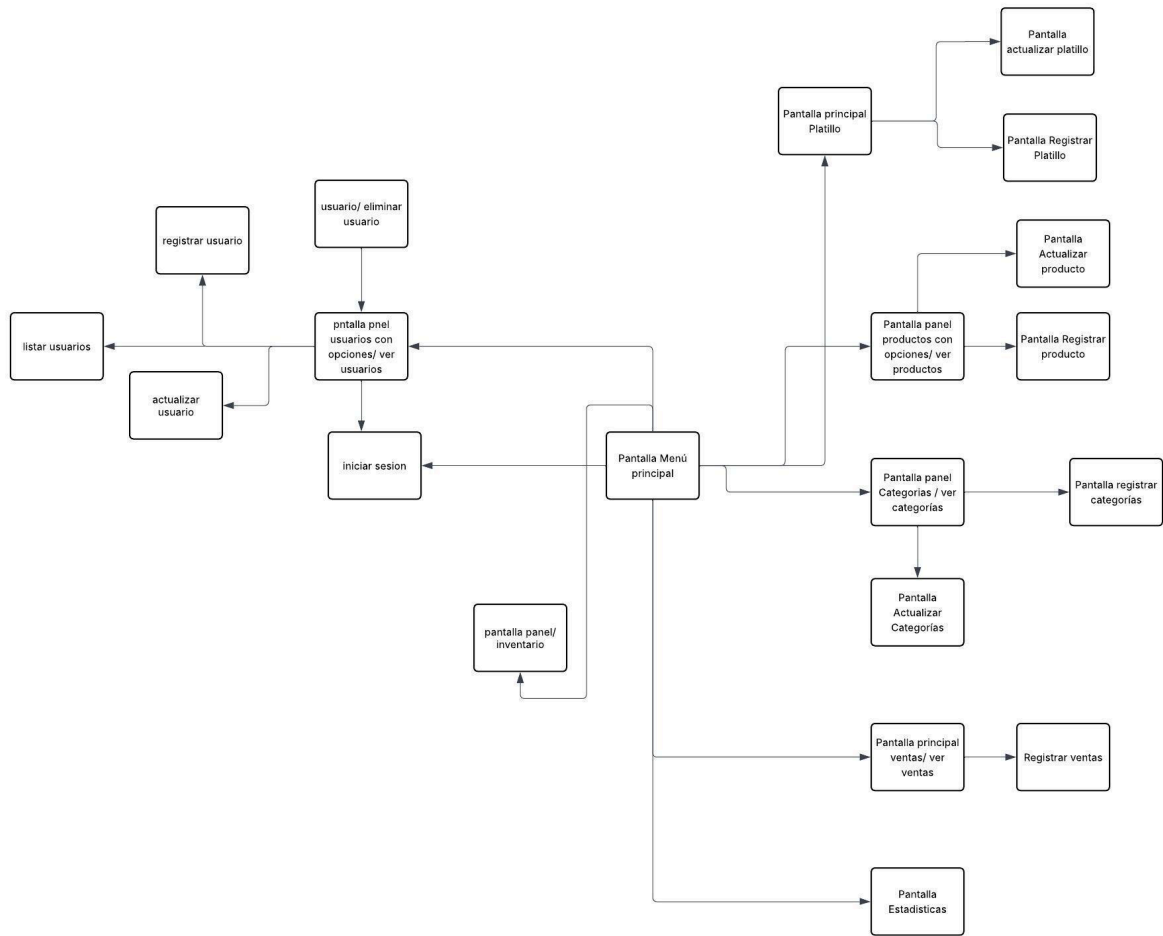
Flujo eliminar/desactivarPlatillo



flujo inventario:

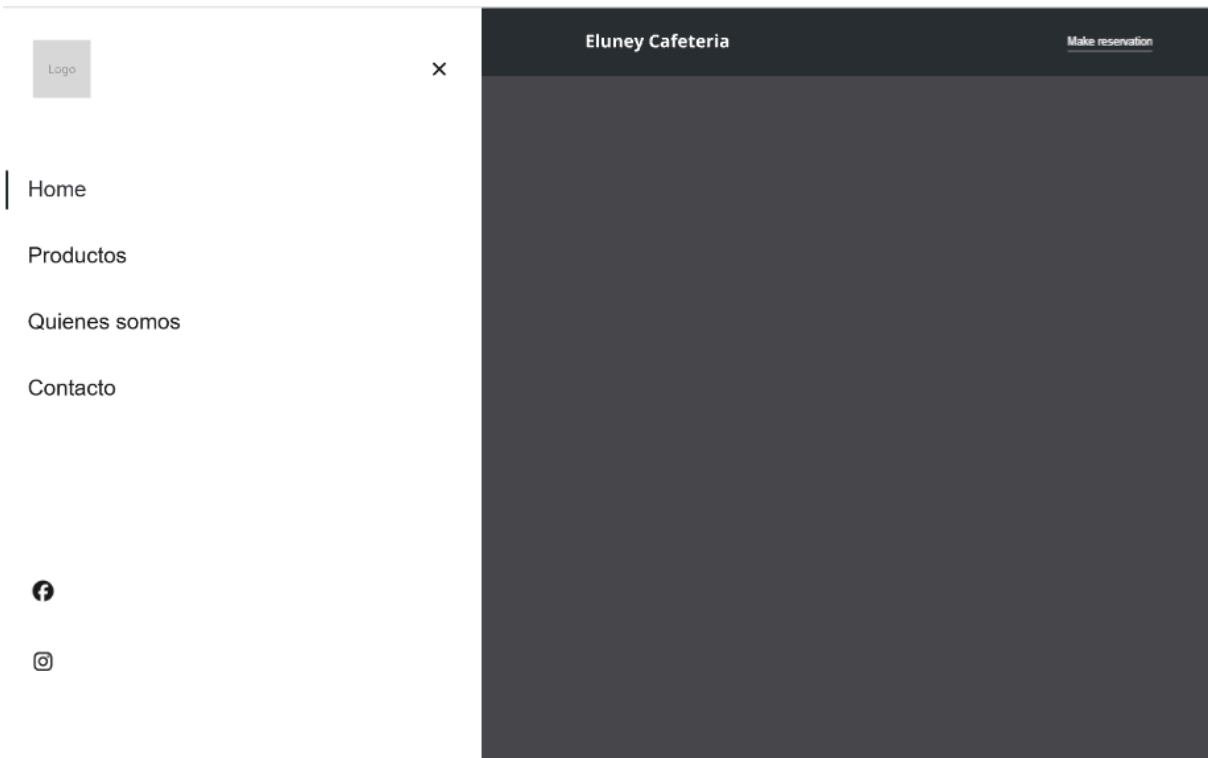


MAPA NAVEGACIÓN PANTALLAS APP ESCRITORIO



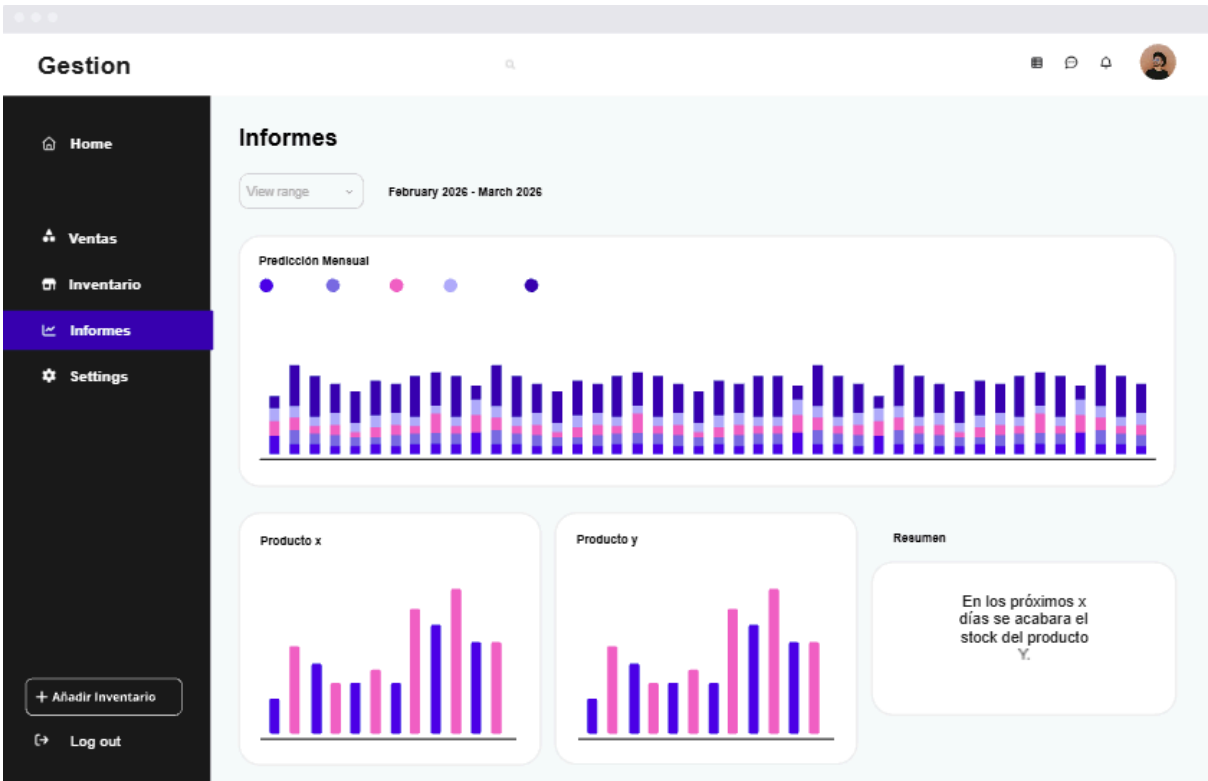
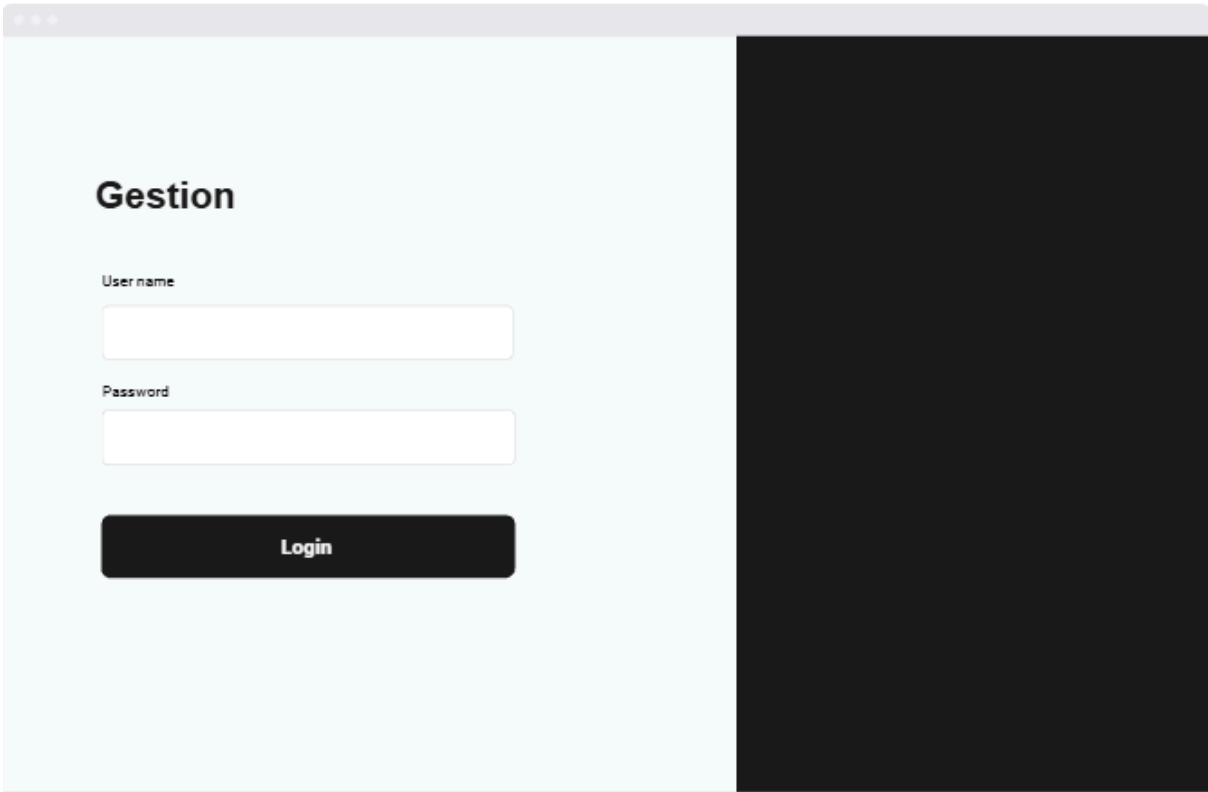
wireframes:

página web



Menu

APPETIZERS		BUNS	
Edamame salad	\$9	Bao Bun Sliders	\$9
pickled green beans, mixed greens, edamame, sesame seeds, ginger ale dressing		bun buns, Bao Bun Sliders, pickled green beans, ginger ale dressing	
Tuna tartare tacos	\$19	Truffle Shiitake Dim Sum	\$11
Sweet onion, avocado, miso dressing, shiso, scallions, chili oil		Luscious truffle porcini shiitake multi cream, white truffle	
Crispy Tempura Veggies	\$12	Peking Duck Bao	\$18
Zucchini, bell pepper, sweet potato, cornstarch batter, chili powder		Bun buns, Peking duck slices, fresh sauce, cucumber	
Sakura Spring Rolls	\$13	Shrimp Har Gow	\$16
Rice paper, cornstarch, noodles, cucumber, carrots, hoisin sauce		Rice paper, shrimp, cornstarch, cucumber, carrot, hoisin sauce	
Seared Scallop Carpaccio	\$20	Veggie Bao Trio	\$15
Fresh scallops, citrus, soy marinade, cashew, olive oil, kani		Bun buns, truffle tofu, mushroom, porcini, mushroom	



Gestion

Home

Ventas

Inventario

Informes

Settings

+ Añadir inventario

Log out

Añadir Inventario

Name*

Item code*

Description

Stock size*

Store availability*

Product photos*

Category*

Price*

Save product

Gestion

Home

Ventas

Inventario

Informes

Settings

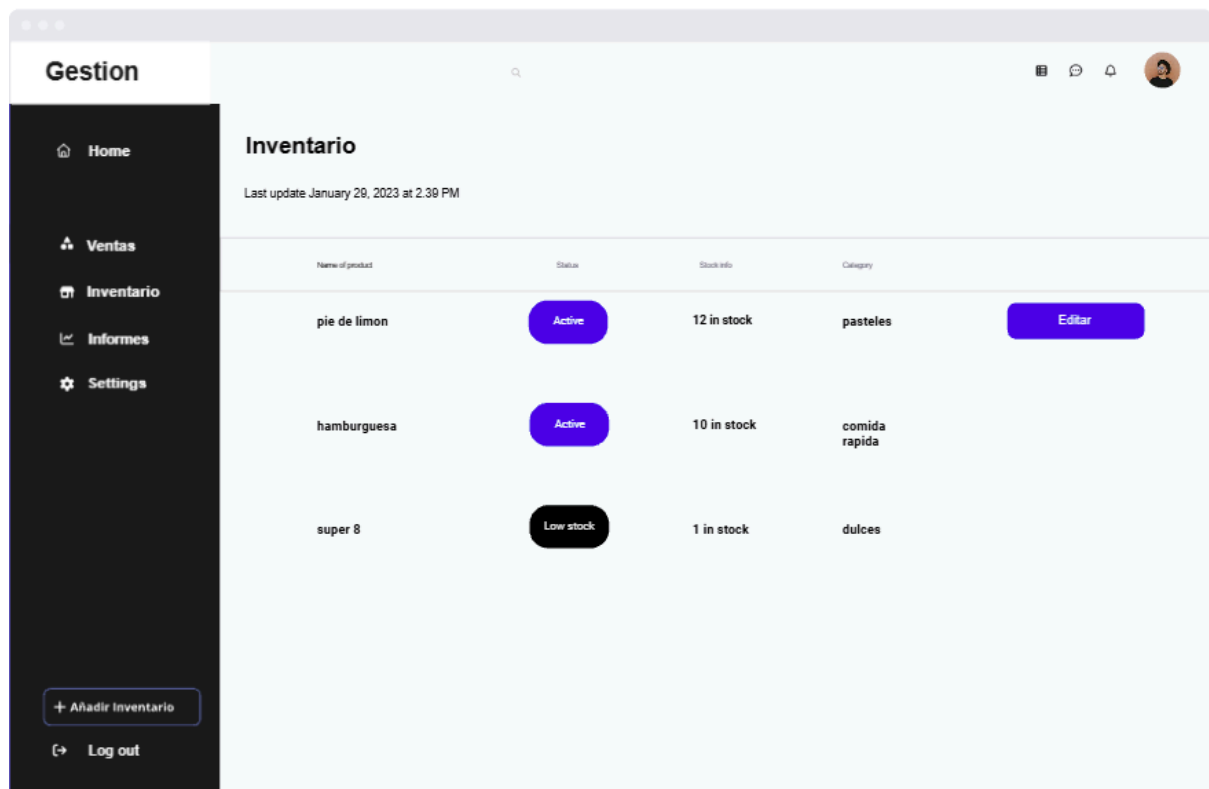
+ Añadir inventario

Log out

Registro de Ventas

Nombre Producto	Moneda	Tipo de pago	Description
pastel 1	10 in stock	T- shirts	4 stores
Pastel 2	10 in stock	T- shirts	4 stores
sandwich 1	28 in stock	food	3 stores
sandwich 2	12 in stock	Outerwear	3 stores

Total venta: 50,000



Lista de Casos de Prueba

Módulo: Productos (Componente: ProductoService)

- **TC-01: Camino Feliz**
 - **Descripción del Escenario:** Registro exitoso de un producto con todos sus parámetros correctos.
 - **Resultado Esperado:** El flujo finaliza sin errores, confirmando el guardado del producto y su respectivo registro en la Auditoría.
 - **Estado Actual:** Listo.
- **TC-02: Protección de Margen**
 - **Descripción del Escenario:** Intento de registro con un margen de ganancia inferior al 10% mínimo.
 - **Resultado Esperado:** Lanza Exception: "Protección de Margen: El precio de venta debe ser al menos un 10% mayor al precio de compra."
 - **Estado Actual:** Listo.
- **TC-03: Producto Nulo**
 - **Descripción del Escenario:** Intento de registrar un objeto Producto que no ha sido instanciado.

- **Resultado Esperado:** Lanza Exception: "El producto no puede ser nulo". Se detiene el sistema antes de interactuar con la base de datos.
- **Estado Actual:** Listo.

- **TC-04: Nombre Vacío**
 - **Descripción del Escenario:** Intento de registro omitiendo el nombre o ingresando únicamente espacios en blanco.
 - **Resultado Esperado:** Lanza Exception: "El nombre del producto es obligatorio.". El flujo se bloquea de inmediato.
 - **Estado Actual:** Listo.

- **TC-05: Precio de Compra Inválido**
 - **Descripción del Escenario:** Intento de registro ingresando un precio de compra igual o menor a cero.
 - **Resultado Esperado:** Lanza Exception: "El precio de compra debe ser mayor a cero". El flujo se bloquea de inmediato.
 - **Estado Actual:** Listo.

- **TC-06: Líquidos/Gramos sin Cantidad**
 - **Descripción del Escenario:** Registro de productos medidos en gramos o mililitros con su valor de cantidad en cero.
 - **Resultado Esperado:** Lanza Exception: "La cantidad debe ser mayor a cero para unidades de medida en gramos o mililitros."
 - **Estado Actual:** Listo.

- **TC-07: Stock Actual Inválido**
 - **Descripción del Escenario:** Intento de registro configurando un stock inicial menor o igual a cero.
 - **Resultado Esperado:** Lanza Exception: "El stock actual debe ser mayor a cero". El flujo se bloquea de inmediato.
 - **Estado Actual:** Listo.

- **TC-08: Stock Mínimo Inválido**
 - **Descripción del Escenario:** Intento de registro configurando un stock de alerta menor o igual a cero.
 - **Resultado Esperado:** Lanza Exception: "El stock mínimo debe ser mayor a cero". El flujo se bloquea de inmediato.
 - **Estado Actual:** Listo.

- **TC-09: Categoría Ausente**
 - **Descripción del Escenario:** Intento de registrar un producto sin asociarlo a ninguna categoría del sistema.
 - **Resultado Esperado:** Lanza Exception: "El producto debe tener una categoría asignada". El flujo se bloquea de inmediato.
 - **Estado Actual:** Listo.

- **TC-10: Nombre Duplicado**
 - **Descripción del Escenario:** Intento de registro de un producto con un nombre idéntico a uno ya existente en el sistema.
 - **Resultado Esperado:** Lanza Exception: "Validación fallida: Ya existe un registro con el nombre...". El sistema impide que se dupliquen datos.
 - **Estado Actual:** Listo.

- **TC-11: Producto Solo Inventario**
 - **Descripción del Escenario:** Validación de que el precio de venta se fuerza a cero de forma automática.
 - **Resultado Esperado:** El flujo finaliza con éxito, pero se valida internamente que el precio de venta del producto fue modificado a 0.0.
 - **Estado Actual:** Listo.

- **TC-12: Error Interno de Almacenamiento**
 - **Descripción del Escenario:** Los datos del producto están perfectos pero el almacenamiento en SQLite falla.
 - **Resultado Esperado:** Lanza Exception: "Error interno: No se pudo guardar el producto en la base de datos.". Se impide la creación de auditoría huérfana.
 - **Estado Actual:** Listo.

Métodos de Consulta y Reportes de Productos

- **TC-13: Filtro Correcto (buscarPorTipoProducto)**
 - **Descripción del Escenario:** Búsqueda pasándole un tipo de producto válido (ej. DIRECTO).
 - **Resultado Esperado:** Retorna la lista de productos que pertenecen exclusivamente a ese tipo.
 - **Estado Actual:** Listo.

●

- **TC-14: Filtro Nulo (buscarPorTipoProducto)**
 - **Descripción del Escenario:** Intento de filtrar productos enviando el tipo como null.
 - **Resultado Esperado:** Lanza Exception: "Debe seleccionar un tipo de producto." El flujo se bloquea.
 - **Estado Actual:** Listo.

- **TC-15: Alerta de Stock (obtenerStockCritico)**
 - **Descripción del Escenario:** Recuperación de productos con existencias por debajo del mínimo.
 - **Resultado Esperado:** Retorna la lista de productos ordenada de forma ascendente (menor a mayor stock).
 - **Estado Actual:** Listo

- **TC-16: Sin Alertas (obtenerStockCritico)**
 - **Descripción del Escenario:** El repositorio devuelve un valor nulo (null) al no haber registros críticos.
 - **Resultado Esperado:** El sistema tolera el nulo y devuelve de forma segura una lista vacía `new ArrayList<>()`.
 - **Estado Actual:** Listo.

Métodos de Modificación y Eliminación de Productos

- **TC-17: Actualización Exitosa (actualizarProducto)**
 - **Descripción del Escenario:** Modificación de datos de un producto válido existente.
 - **Resultado Esperado:** Modifica los datos en la base de datos, registra el evento en la Auditoría y retorna true.
 - **Estado Actual:** Listo.

- **TC-18: Producto Nulo (actualizarProducto)**
 - **Descripción del Escenario:** Intento de actualizar enviando el objeto como null.
 - **Resultado Esperado:** Lanza Exception: "El producto no puede ser nulo".
 - **Estado Actual:** Listo.

- **TC-19: Nombre Vacío (actualizarProducto)**
 - **Descripción del Escenario:** Intento de actualizar el producto dejando el nombre en blanco o con espacios.
 - **Resultado Esperado:** Lanza Exception: "El nombre del producto es obligatorio."
 - **Estado Actual:** Listo.

- **TC-20: Nombre Duplicado (actualizarProducto)**
 - **Descripción del Escenario:** Intento de renombrar el producto usando el nombre de otro ya existente.
 - **Resultado Esperado:** Lanza Exception: "Ya existe un producto con el nombre '...'".
 - **Estado Actual:** Listo.

- **TC-21: Precio de Compra Inválido (actualizarProducto)**
 - **Descripción del Escenario:** Actualizar asignando un precio de compra igual o menor a cero.
 - **Resultado Esperado:** El método detiene el proceso de forma segura y retorna un valor false.
 - **Estado Actual:** Listo.

- **TC-22: Fallo de Persistencia (actualizarProducto)**
 - **Descripción del Escenario:** Los datos son correctos pero el repositorio no encuentra el ID o falla.
 - **Resultado Esperado:** Lanza Exception: "Error al actualizar el producto. Verifique que el producto exista..."
 - **Estado Actual:** Listo.

- **TC-23: ID Inválido (eliminarProducto)**
 - **Descripción del Escenario:** Intento de eliminación enviando un ID menor o igual a cero.
 - **Resultado Esperado:** Lanza Exception: "ID de producto no válido."
 - **Estado Actual:** Listo.

- **TC-24: Producto Inexistente (eliminarProducto)**
 - **Descripción del Escenario:** Intento de eliminar un ID que no se encuentra en el sistema.
 - **Resultado Esperado:** Lanza Exception: "El producto no existe".

- **Estado Actual:** Listo.
- **TC-25: Producto con Dependencias (eliminarProducto)**
 - **Descripción del Escenario:** El producto está amarrado a una venta o platillo activo.
 - **Resultado Esperado:** No se elimina de la BD; se ejecuta un borrado lógico (cambio a inactivo) y retorna el texto descriptivo.
 - **Estado Actual:** Listo.
- **TC-26: Eliminación Física Exitosa (eliminarProducto)**
 - **Descripción del Escenario:** El producto no tiene amarres y se puede borrar por completo.
 - **Resultado Esperado:** Se elimina permanentemente de la BD, registra la Auditoría de "ELIMINACION" y retorna "ELIMINADO".
 - **Estado Actual:** Listo.

II. Módulo de Categorías (Componente: CategoriaService)

- **TC-27: Registro Exitoso (registrarCategoria)**
 - **Descripción del Escenario:** Inserción de una categoría nueva con nombre y descripción válidos.
 - **Resultado Esperado:** Se guarda en la BD aplicando un .trim() al nombre/descripción y genera auditoría en el módulo CATEGORIAS.
 - **Estado Actual:** Listo.
- **TC-28: Categoría Nula (registrarCategoria)**
 - **Descripción del Escenario:** Intento de registrar un objeto de categoría que no está instanciado.
 - **Resultado Esperado:** Lanza Exception: "La categoría no puede ser nula.".
 - **Estado Actual:** Listo.
- **TC-29: Nombre Obligatorio (registrarCategoria)**
 - **Descripción del Escenario:** Intento de registrar una categoría sin nombre o con espacios en blanco.
 - **Resultado Esperado:** Lanza Exception: "El nombre de la categoría es obligatorio.".
 - **Estado Actual:** Listo.

- **TC-30: Categoría Duplicada (registrarCategoría)**
 - **Descripción del Escenario:** Intento de guardar una categoría cuyo nombre exacto ya existe.
 - **Resultado Esperado:** Lanza Exception: "La categoría ya existe."
 - **Estado Actual:** Listo.

- **TC-31: Actualización Exitosa (actualizarCategoría)**
 - **Descripción del Escenario:** Modificación correcta de los datos de una categoría válida.
 - **Resultado Esperado:** Aplica los cambios, guarda la Auditoría de "ACTUALIZACION" y retorna true.
 - **Estado Actual:** Listo.

- **TC-32: ID o Categoría Inválida (actualizarCategoría)**
 - **Descripción del Escenario:** Objeto nulo o con un ID de registro menor o igual a cero.
 - **Resultado Esperado:** Lanza Exception: "Categoría no válida."
 - **Estado Actual:** Listo.

- **TC-33: Nombre Vacío en Edición (actualizarCategoría)**
 - **Descripción del Escenario:** Dejar el nombre de la categoría vacío durante la edición.
 - **Resultado Esperado:** Lanza Exception: "El nombre de la categoría es obligatorio."
 - **Estado Actual:** Listo.

- **TC-34: ID Inválido (eliminarCategoría)**
 - **Descripción del Escenario:** Intento de remover una categoría enviando un ID igual o menor a cero.
 - **Resultado Esperado:** Lanza Exception: "ID de categoría no válido."
 - **Estado Actual:** Listo.

- **TC-35: Categoría en Uso (eliminarCategoría)**
 - **Descripción del Escenario:** Intento de borrar una categoría que ya contiene productos registrados.
 - **Resultado Esperado:** Lanza Exception: "No se puede eliminar la categoría". Protege la integridad de los datos.
 - **Estado Actual:** Listo.

- **TC-36: Eliminación Exitosa (eliminarCategoria)**
 - **Descripción del Escenario:** Categoría vacía y sin relaciones se borra de la base de datos.
 - **Resultado Esperado:** Se remueve físicamente de SQLite, genera la Auditoría de "ELIMINACION" y retorna true.
 - **Estado Actual:** Listo.

Módulo: Platillos (Componente: PlatilloService)

- **TC-01: Registrar platillo exitoso**
 - **Comportamiento Esperado:** No lanza ninguna excepción (assertDoesNotThrow), permitiendo el flujo normal de registro.
 - **Estado Actual:** Listo.
- **TC-02: Registrar platillo nulo**
 - **Comportamiento Esperado:** Lanza excepción con el mensaje exacto: "El platillo no puede ser nulo".
 - **Estado Actual:** Listo.
- **TC-03: Registrar platillo con nombre vacío**
 - **Comportamiento Esperado:** Lanza excepción con el mensaje exacto: "El nombre del platillo es obligatorio".
 - **Estado Actual:** Listo.
- **TC-04: Registrar platillo con precio inválido**
 - **Comportamiento Esperado:** Lanza excepción con el mensaje exacto: "El platillo debe tener un precio de venta mayor a cero".
 - **Estado Actual:** Listo.
- **TC-05: Registrar platillo con nombre duplicado**
 - **Comportamiento Esperado:** Lanza excepción con el mensaje exacto: "Ya existe un platillo con el nombre '...'".
 - **Estado Actual:** Listo.
- **TC-06: Registrar platillo con tipo incorrecto**
 - **Comportamiento Esperado:** Lanza excepción con el mensaje exacto: "El tipo de producto debe ser PLATILLO para registrar un platillo".
 - **Estado Actual:** Listo.

- **TC-07: Calcular costo ingrediente por Unidades**
 - **Comportamiento Esperado:** El retorno del método debe ser el producto exacto entre el precio de compra y las unidades utilizadas.
 - **Estado Actual:** Listo.

- **TC-08: Calcular costo ingrediente por gramos**
 - **Comportamiento Esperado:** El retorno debe realizar la conversión matemática proporcional basándose en un costo por kilo (1000g).
 - **Estado Actual:** Listo.

- **TC-09: Calcular costo de producción total**
 - **Comportamiento Esperado:** El atributo costoProduccion del platillo se modifica internamente con la suma exacta de todos sus ingredientes asociados.
 - **Estado Actual:** Listo.

- **TC-10: Validación de stock exitosa**
 - **Comportamiento Esperado:** No lanza excepciones (assertDoesNotThrow), confirmando que la despensa cubre los requerimientos.
 - **Estado Actual:** Listo.

- **TC-11: Validación stock insuficiente (Unidades)**
 - **Comportamiento Esperado:** Lanza la excepción respectiva por no tener suficientes piezas en el campo stockActual.
 - **Estado Actual:** Listo.

- **TC-12: Validación stock insuficiente (Gramos)**
 - **Comportamiento Esperado:** Lanza la excepción respectiva por falta de miligramos/gramos necesarios evaluados en el campo cantidad.
 - **Estado Actual:** Listo.

- **TC-13: Calcular stock en tiempo real (Limitante)**
 - **Comportamiento Esperado:** Retorna un número entero que corresponde estrictamente al ingrediente más escaso (el cuello de botella de la receta).
 - **Estado Actual:** Listo.

- **TC-14: Calcular stock en tiempo real es cero**
 - **Comportamiento Esperado:** Retorna el entero 0 al detectar que falta al menos un ingrediente obligatorio para armar el plato.
 - **Estado Actual:** Listo.

- **TC-15: Actualizar platillo exitoso**
 - **Comportamiento Esperado:** Modifica el registro de forma correcta, recalculando costos previamente y guardando una auditoría tipo "ACTUALIZACION".
 - **Estado Actual:** Listo.

- **TC-16: Eliminar platillo sin ventas (Físico)**
 - **Comportamiento Esperado:** Retorna el String "ELIMINADO" y procede con un borrado completo y físico del registro en la base de datos.
 - **Estado Actual:** Listo.

- **TC-17: Eliminar platillo con ventas (Lógico)**
 - **Comportamiento Esperado:** Retorna el String "Este platillo está asociado a una venta y fue desactivado." resguardando la integridad histórica mediante un cambio de estado.
 - **Estado Actual:** Listo.

Módulo: Inteligencia de Negocio y Análisis Estadístico (Componente: EstadisticaService)

- **TC-01: Camino Feliz (Ingresos)**
 - **Descripción del Escenario:** Consulta exitosa de los ingresos financieros totales enviando un periodo válido ("MES").
 - **Resultado Esperado:** El flujo finaliza sin errores, retornando el valor numérico decimal exacto calculado por la base de datos.
 - **Estado Actual:** Listo.

- **TC-02: Periodo Inválido / Nulo**
 - **Descripción del Escenario:** Intento de obtener ingresos enviando un parámetro de tiempo nulo o con espacios en blanco.
 - **Resultado Esperado:** El sistema intercepta el error preventivamente y retorna 0.0 sin realizar peticiones costosas a los repositorios.
 - **Estado Actual:** Listo.

- **TC-03: Balance Financiero Exitoso**

- **Descripción del Escenario:** Cálculo de utilidades operacionales cruzando ingresos positivos y egresos existentes.
 - **Resultado Esperado:** Retorna una entidad BalanceFinancieroDTO con la utilidad neta exacta y el margen porcentual bien calculado.
 - **Estado Actual:** Listo.
- **TC-04: Protección División por Cero**
 - **Descripción del Escenario:** Intento de calcular el balance cuando la empresa registra ingresos equivalentes a 0.0.
 - **Resultado Esperado:** Evita desbordamientos lógicos (NaN / Infinity), forzando el margen a cero y arrojando pérdida neta en la utilidad.
 - **Estado Actual:** Listo.
- **TC-05: Ranking de Productos**
 - **Descripción del Escenario:** Extracción masiva del top de artículos más vendidos durante el rango de tiempo seleccionado.
 - **Resultado Esperado:** Procesa la información y devuelve una colección ordenada de objetos RankingProductosDTO listos para la interfaz.
 - **Estado Actual:** Listo.
- **TC-06: Ranking de Desempeño Comercial**
 - **Descripción del Escenario:** Consulta del rendimiento analítico y volumen de ventas asignado a cada empleado.
 - **Resultado Esperado:** Retorna una lista estructurada de RankingVendedoresDTO emparejada con las métricas del repositorio.
 - **Estado Actual:** Listo.
- **TC-07: Algoritmo de Riesgo Crítico (Stock)**
 - **Descripción del Escenario:** Proyección donde un insumo tiene stock bajo y alta demanda diaria (Días < 3).
 - **Resultado Esperado:** El sistema asigna un indiceRiesgo estricto de 0.9 y calcula una cantidad de compra sugerida mayor a cero.
 - **Estado Actual:** Listo.
- **TC-08: Algoritmo de Riesgo Alto (Stock)**
 - **Descripción del Escenario:** Proyección donde el tiempo matemático calculado para el desabastecimiento está entre 3 y 7 días.
 - **Resultado Esperado:** El algoritmo evalúa las variables y clasifica la alerta fijando el indiceRiesgo exactamente en 0.6.

- **Estado Actual:** Listo.

- **TC-09: Algoritmo de Riesgo Preventivo**
 - **Descripción del Escenario:** Proyección donde el tiempo de agotamiento estimado se sitúa en el rango de 7 a 14 días totales.
 - **Resultado Esperado:** El algoritmo clasifica la métrica en un estado preventivo de atención, fijando el indiceRiesgo en 0.3.
 - **Estado Actual:** Listo.

- **TC-10: Algoritmo de Riesgo Bajo**
 - **Descripción del Escenario:** Proyección donde las existencias de almacén son sanas y el agotamiento supera los 14 días.
 - **Resultado Esperado:** El sistema asigna un indiceRiesgo de 0.1 y fuerza el parámetro de cantidadSugerida a 0 (Evita sobre stock).
 - **Estado Actual:** Listo.

- **TC-11: Control de Demanda Nula**
 - **Descripción del Escenario:** Análisis predictivo de un producto registrado en catálogo que no posee ninguna venta histórica.
 - **Resultado Esperado:** El software asigna por defecto 999 días de holgura, fija riesgo bajo y bloquea divisiones indeterminadas por cero.
 - **Estado Actual:** Listo.

- **TC-12: Redondeo de Abastecimiento**
 - **Descripción del Escenario:** Escenario de riesgo medio donde la fórmula matemática de compra da como resultado un valor fraccionario.
 - **Resultado Esperado:** Aplica un redondeo hacia arriba (Math.ceil) para asegurar el suministro de al menos 1 unidad entera del artículo.
 - **Estado Actual:** Listo.

- **TC-13: Colecciones Vacías**
 - **Descripción del Escenario:** El inventario general de la base de datos se encuentra completamente vacío al momento del cálculo masivo.
 - **Resultado Esperado:** El método responde de forma segura con una lista vacía, impidiendo fallos catastróficos por puntero nulo (NPE).
 - **Estado Actual:** Listo.

- **TC-14: Trazabilidad de Auditoría Batch**

- **Descripción del Escenario:** Ejecución del proceso analítico predictivo por lotes (Batch) en condiciones normales.
 - **Resultado Esperado:** Verifica la correcta invocación del método registrarEvento guardando en la persistencia el mensaje técnico de lote.
 - **Estado Actual:** Listo.
- **TC-15: Resiliencia ante Fallos Críticos**
 - **Descripción del Escenario:** El motor de la base de datos (SQLite) sufre una desconexión o interrupción (SQLException).
 - **Resultado Esperado:** El service encapsula internamente la excepción, desvía el error a la consola de diagnóstico y evita que la aplicación se caiga.
 - **Estado Actual:** Listo.

Módulo: Ventas (Componente: VentaService)

Método: **resolverItemsVenta**

TC-01: Asignación de Ítems Mixtos

- **Descripción del Escenario:** Se ingresa una cadena de texto que contiene tanto productos como platillos válidos presentes en los catálogos.
- **Resultado Esperado:** Los ítems mixtos se identifican y se asignan correctamente a sus respectivas listas (**detalleVentas** y **detallePlatillos**) dentro del objeto **ventaAplicacion**.
- **Estado Actual:** Listo.

TC-02: Input Nulo

- **Descripción del Escenario:** Intento de resolver ítems de venta enviando un valor **null** como entrada.
- **Resultado Esperado:** Lanza **Exception** con un mensaje que contiene "no puede estar vacío".
- **Estado Actual:** Listo.

TC-03: Input Vacío

- **Descripción del Escenario:** Intento de resolver ítems enviando una cadena de texto vacía ("").
- **Resultado Esperado:** Lanza **Exception**.
- **Estado Actual:** Listo.

TC-04: Ítem No Encontrado

- **Descripción del Escenario:** Se ingresa una cadena donde al menos uno de los ítems no existe en ninguno de los catálogos proporcionados.
- **Resultado Esperado:** Lanza **Exception** y el mensaje de error incluye el nombre exacto del ítem no encontrado.
- **Estado Actual:** Listo.

TC-05: Catálogos Vacíos

- **Descripción del Escenario:** Intento de resolver un ítem válido, pero los catálogos de productos y platillos proporcionados no contienen elementos (están vacíos).
- **Resultado Esperado:** Lanza **Exception** indicando que el ítem no fue encontrado.
- **Estado Actual:** Listo.

Método: **calcularTotal**

TC-06: Suma de Productos y Platillos

- **Descripción del Escenario:** Se ingresan ítems mixtos con sus catálogos respectivos para calcular el total de la venta.
- **Resultado Esperado:** La suma total incluye correctamente los precios tanto de los productos como de los platillos solicitados.
- **Estado Actual:** Listo.

TC-07: Cálculo con Input Nulo

- **Descripción del Escenario:** Se envía un valor **null** en lugar de la cadena de ítems a calcular.
- **Resultado Esperado:** Retorna **0.0** sin lanzar una excepción **NullPointerException**.
- **Estado Actual:** Listo.

TC-08: Catálogo de Productos Nulo

- **Descripción del Escenario:** Se envía **null** como catálogo de productos, calculando únicamente el valor de los platillos.
- **Resultado Esperado:** El sistema tolera el catálogo nulo sin lanzar excepciones y calcula correctamente el total usando solo los platillos.
- **Estado Actual:** Listo.

TC-09: Cálculo con Ítem Inexistente

- **Descripción del Escenario:** Se intenta calcular el total de un ítem que no tiene coincidencia en los catálogos.
- **Resultado Esperado:** El ítem es ignorado y no suma valor al cálculo, retornando **0.0**.
- **Estado Actual:** Listo.

Método: **procesarNuevaVentaApp**

TC-10: Total No Numérico

- **Descripción del Escenario:** Se ingresa un valor de "total" con un formato inválido (ej. caracteres alfabéticos).
- **Resultado Esperado:** Lanza **Exception** con un mensaje indicando error de "formato numérico".
- **Estado Actual:** Listo.

TC-11: Construcción con Datos Válidos

- **Descripción del Escenario:** Registro de una nueva venta enviando todos los datos correctos y con una sesión de usuario activa.
- **Resultado Esperado:** Se construye y retorna un objeto **ventaAplicacion** con todos sus campos y listas asignados correctamente (total, fecha, tipo de pago, descripción, usuario, etc.).
- **Estado Actual:** Listo.

Método: **descontarProductoYPlatillo**

TC-12: Descuento de Producto con Stock Suficiente

- **Descripción del Escenario:** Venta de un producto que tiene un stock actual mayor a cero.
- **Resultado Esperado:** Se descuenta exactamente 1 unidad mediante el repositorio (**actualizarStockFisico**).
- **Estado Actual:** Listo.

TC-13: Prevención de Stock Negativo (Sin Stock)

- **Descripción del Escenario:** Venta de un producto cuyo stock actual es 0.
- **Resultado Esperado:** No se ejecuta ningún descuento en la base de datos, evitando que el stock quede en negativo.
- **Estado Actual:** Listo.

TC-14: Descuento de Ingrediente tipo UNIDAD

- **Descripción del Escenario:** Venta de un platillo cuyo ingrediente se mide por unidades y cuenta con stock suficiente.
- **Resultado Esperado:** Se descuenta la cantidad física exacta especificada en la receta del platillo.
- **Estado Actual:** Listo.

TC-15: Ingrediente tipo UNIDAD sin Stock Suficiente

- **Descripción del Escenario:** Venta de un platillo donde el ingrediente en unidades tiene menor stock disponible que el requerido por la receta.
- **Resultado Esperado:** No se ejecuta ningún descuento.
- **Estado Actual:** Listo.

TC-16: Ingrediente tipo GRAMOS con Rollover

- **Descripción del Escenario:** Venta de un platillo donde el ingrediente requerido supera la cantidad fraccionaria actual, requiriendo abrir una nueva unidad física.

- **Resultado Esperado:** Se descuenta 1 unidad del stock físico y se ajusta la cantidad fraccionaria según la cantidad por defecto del producto.
- **Estado Actual:** Listo.

TC-17: Ingrediente tipo GRAMOS sin Rollover

- **Descripción del Escenario:** Venta de un platillo donde el ingrediente tiene suficiente cantidad fraccionaria disponible sin necesidad de abrir una nueva unidad física.
- **Resultado Esperado:** Sólo se ajusta la cantidad fraccionaria sin descontar del stock físico entero.
- **Estado Actual:** Listo.

TC-18: Ingrediente Nulo en Receta

- **Descripción del Escenario:** Procesamiento de un platillo que contiene un `DetallePlatillo` con un producto nulo.
- **Resultado Esperado:** Se ignora el detalle sin interactuar con los repositorios y no se lanzan excepciones (`NullPointerException`).
- **Estado Actual:** Listo.

TC-19: Platillo sin Ingredientes

- **Descripción del Escenario:** Se procesa un platillo cuya lista de ingredientes es nula.
- **Resultado Esperado:** El flujo finaliza sin interactuar con el repositorio de movimientos.
- **Estado Actual:** Listo.

Funciones CRUD y de Repositorio

TC-20: Traer Ventas (Lista con Datos)

- **Descripción del Escenario:** Solicitud para obtener todas las ventas cuando existen registros en base de datos.
- **Resultado Esperado:** Retorna la lista de `ventaAplicacion` recibida directamente desde el repositorio.
- **Estado Actual:** Listo.

TC-21: Traer Ventas (Lista Vacía)

- **Descripción del Escenario:** Solicitud para obtener todas las ventas cuando el repositorio no tiene registros.
- **Resultado Esperado:** La lista vacía se propaga correctamente sin generar errores.
- **Estado Actual:** Listo.

TC-22: Deduplicación de Fechas

- **Descripción del Escenario:** Solicitud de las fechas de venta cuando existen múltiples registros realizados el mismo día pero a distinta hora.

- **Resultado Esperado:** Las fechas se agrupan y deduplican, mostrando solo un elemento representativo por día (yyyy-MM-dd).
- **Estado Actual:** Listo.

TC-23: Orden de Fechas Descendente

- **Descripción del Escenario:** Solicitud de fechas de ventas realizadas en distintos días.
- **Resultado Esperado:** La lista devuelta está ordenada de manera descendente (la fecha más reciente aparece primero).
- **Estado Actual:** Listo.

TC-24: Fallo en Repositorio de Fechas

- **Descripción del Escenario:** Excepción a nivel de base de datos al intentar consultar las fechas de venta.
- **Resultado Esperado:** El servicio captura el error y retorna `null` sin propagar la excepción hacia el usuario.
- **Estado Actual:** Listo.

TC-25: Formateo de Fecha para Búsqueda (LIKE)

- **Descripción del Escenario:** Solicitud de obtención de ventas filtradas por una fecha específica.
- **Resultado Esperado:** La cadena de fecha se formatea correctamente envolviéndose en porcentajes (%fecha%) antes de consultar al repositorio.
- **Estado Actual:** Listo.

TC-26: Retorno de Ventas por Fecha como ObservableList

- **Descripción del Escenario:** Búsqueda exitosa de ventas por fecha.
- **Resultado Esperado:** Convierte y retorna correctamente el resultado del repositorio en formato `ObservableList<ventaAplicacion>`.
- **Estado Actual:** Listo.

TC-27: Fallo en Repositorio al Buscar por Fecha

- **Descripción del Escenario:** Excepción en base de datos al buscar las ventas de un día específico.
- **Resultado Esperado:** El servicio captura la excepción y retorna `null`.
- **Estado Actual:** Listo.

TC-28: Subida de Tabla a BD (Éxito)

- **Descripción del Escenario:** Se envía una tabla completa de ventas y el repositorio confirma el guardado.
- **Resultado Esperado:** Retorna `true`.
- **Estado Actual:** Listo.

TC-29: Subida de Tabla a BD (Fallo)

- **Descripción del Escenario:** El repositorio rechaza o falla al intentar guardar la tabla de ventas.
- **Resultado Esperado:** Retorna **false**.
- **Estado Actual:** Listo.

TC-30: Guardado Simple de Venta (Comportamiento Legacy)

- **Descripción del Escenario:** Solicitud de guardar una venta con sus productos asociados.
- **Resultado Esperado:** Delega al repositorio pasando los datos correspondientes y una lista vacía para platillos por defecto, retornando **true**.
- **Estado Actual:** Listo.

TC-31: Guardado Simple de Venta (Fallo en Repo)

- **Descripción del Escenario:** Error en repositorio al registrar una venta simple.
- **Resultado Esperado:** Propaga correctamente el fallo y retorna **false**.
- **Estado Actual:** Listo.

Test usuarios:

TC-01

- **Descripción del Escenario:** Camino Feliz (Entrada): Intento de registrar correctamente una entrada de mercadería con datos válidos (**TipoMovimiento.ENTRADA**), ID de producto existente y cantidad positiva absoluta.
- **Resultado Esperado:** El flujo finaliza con éxito retornando el objeto **MovimientoInventario** modificado con su fecha y hora local en vivo. Se verifica mediante aserciones que el stock físico se incrementó (signo positivo) y se ejecutó el disparador de alertas de stock.
- **Estado Actual:** Listo para codificar.

TC-02

- **Descripción del Escenario:** Camino Feliz (Salida con Stock): Intento de registrar una salida de mercadería por venta teniendo disponibilidad o existencias físicas suficientes en la despensa del negocio (**RN-INV-01**).
- **Resultado Esperado:** El flujo procesa la solicitud con éxito, transforma de forma interna y automatizada el signo de la cantidad a negativo para restar del inventario, guarda los cambios y retorna el objeto movimiento correspondiente.
- **Estado Actual:** Listo para codificar.

TC-03

- **Descripción del Escenario:** Movimiento Nulo: Intento de invocar el procesamiento de inventario enviando un objeto `MovimientoInventario` completamente nulo al servicio.
- **Resultado Esperado:** El método intercepta el error en su primera compuerta lógica y retorna un valor estrictamente `null` de forma segura, evitando un fallo por `NullPointerException`.
- **Estado Actual:** Listo para codificar.

TC-04

- **Descripción del Escenario:** Falta Tipo de Movimiento: Intento de procesar un registro de inventario donde el atributo `TipoMovimiento` es enviado como nulo o vacío.
- **Resultado Esperado:** El bloque lógico condicional detecta la anomalía, el flujo es desviado hacia la captura general de excepciones, se imprime el error y el método retorna un valor `null`.
- **Estado Actual:** Listo para codificar.

TC-05

- **Descripción del Escenario:** Validación de Stock Insuficiente (Salida Venta): Intento de procesar una `SALIDA_VENTA` cuya cantidad requerida supera el total de unidades devueltas por el método `obtenerStockActual` en la base de datos (`RN-INV-02`).
- **Resultado Esperado:** Se activa la regla de restricción física (Pre-Check), bloqueando la transacción para impedir stocks negativos inválidos, se imprime el mensaje de aviso en consola y el sistema retorna `null`.
- **Estado Actual:** Listo para codificar.

TC-06

- **Descripción del Escenario:** Validación de Stock Insuficiente (Merma): Intento de reportar una `MERMA` de un producto cuyas unidades averiadas o mermadas superan el stock físico real disponible en el sistema.
- **Resultado Esperado:** El método `validarDisponibilidad` retorna `false`, deteniendo inmediatamente la operación del repositorio y devolviendo un valor de retorno `null`.
- **Estado Actual:** Listo para codificar.

TC-07

- **Descripción del Escenario:** Verificación de Inversión de Signo Automatizada: Verificar que al registrar un movimiento de tipo `MERMA`, `SALIDA_VENTA` o `AJUSTE` (cuando corresponda a resta) la cantidad numérica sea multiplicada internamente por `-1`.
- **Resultado Esperado:** Se valida mediante un captor de argumentos (`ArgumentCaptor`) de Mockito que el valor entero final inyectado al método

`actualizarStockFisico` sea estrictamente negativo, garantizando el descuento físico en la tabla de productos.

- **Estado Actual:** Listo para codificar.

TC-08

- **Descripción del Escenario:** Consistencia del Historial (Cantidad Absoluta): Verificar que a pesar de que el stock físico sea alterado con un signo negativo, el registro en el historial de auditoría almacene la cantidad en formato absoluto positivo.
- **Resultado Esperado:** Se comprueba mediante aserciones que el objeto enviado a `registrarMovimiento` contenga el valor positivo absoluto (`Math.abs`), manteniendo registros contables limpios.
- **Estado Actual:** Listo para codificar.

TC-09

- **Descripción del Escenario:** Interrupción por Fallo en Actualización de Stock: Escenario donde la consulta sql o el repositorio `actualizarStockFisico` retorna `false` debido a un bloqueo temporal de la tabla o base de datos SQLite.
- **Resultado Esperado:** El servicio detecta el fallo de consistencia en el primer paso de persistencia, aborta de forma controlada la transacción imprimiendo un mensaje en `System.err` y retorna `null` sin llegar a insertar datos en el historial.
- **Estado Actual:** Listo para codificar.

TC-10

- **Descripción del Escenario:** Interrupción por Fallo en Registro de Historial: Escenario crítico donde el stock en la tabla de productos se descuenta exitosamente (`true`) pero la inserción física subsiguiente en la tabla `historial_inventario` falla retornando `false`.
- **Resultado Esperado:** El servicio intercepta el quiebre de persistencia del historial, frena el flujo regular e impide el retorno del objeto válido devolviendo un valor `null`.
- **Estado Actual:** Listo para codificar.

TC-11

- **Descripción del Escenario:** Verificación de Disparador Automático de Alertas: Procesamiento exitoso de un movimiento regular para corroborar que el servicio siempre consulte los límites de existencias críticas.
- **Resultado Esperado:** El flujo finaliza correctamente y se verifica mediante Mockito que el método de negocio de alertas del repositorio (`movimientoRepo.generarAlertaStock()`) sea invocado exactamente una vez (`times(1)`).
- **Estado Actual:** Listo para codificar.

TC-12

- **Descripción del Escenario:** Manejo Robusto de Excepciones de Base de Datos: Simulación de una falla catastrófica de conexión o una excepción en tiempo de ejecución (`RuntimeException`) lanzada por el driver JDBC de SQLite.
- **Resultado Esperado:** El bloque general `catch (Exception e)` captura el error de infraestructura, imprime la traza del error en la terminal del servidor mediante `e.printStackTrace()` para su posterior diagnóstico y el sistema retorna un valor seguro de `null`.
- **Estado Actual:** Listo para codificar.

Test Inventario

Componente: `InventarioService`

- **TC-01**
 - **Descripción del Escenario:** Camino Feliz (Entrada): Registro correcto de una entrada de mercadería con datos válidos (`TipoMovimiento.ENTRADA`).
 - **Resultado Esperado (Validación / Excepción / Retorno):** El flujo finaliza sin errores (`assertDoesNotThrow`), retornando el objeto `MovimientoInventario` creado con su fecha en vivo. Se verifica la llamada a `actualizarStockFisico` con signo positivo.
 - **Estado Actual:** ¡Listo para codificar!
- **TC-02**
 - **Descripción del Escenario:** Camino Feliz (Salida con Stock): Registro de una salida de mercadería teniendo disponibilidad suficiente en la despensa (RN-INV-01).
 - **Resultado Esperado (Validación / Excepción / Retorno):** El flujo procesa la solicitud, transforma el signo de la cantidad a negativo para la actualización de stock y retorna el objeto movimiento modificado de forma exitosa.
 - **Estado Actual:** ¡Listo para codificar!
- **TC-03**
 - **Descripción del Escenario:** Movimiento Nulo: Intento de procesar un objeto `MovimientoInventario` que llega completamente nulo al servicio.
 - **Resultado Esperado (Validación / Excepción / Retorno):** Intercepción inmediata en la primera compuerta lógica, retornando estrictamente un valor `null` para evitar un `NullPointerException`.
 - **Estado Actual:** ¡Listo para codificar!
- **TC-04**
 - **Descripción del Escenario:** Falta Tipo de Movimiento: Intento de procesar un movimiento donde el atributo `TipoMovimiento` es nulo.

- **Resultado Esperado (Validación / Excepción / Retorno):** El servicio falla al evaluar la condición en el bloque `switch` o `if`, disparando la captura de excepción en el bloque `catch` y retornando `null`.
- **Estado Actual:** ¡Listo para codificar!
- **TC-05**
 - **Descripción del Escenario:** Stock Insuficiente (Salida Venta): Intento de procesar una `SALIDA_VENTA` cuya cantidad requerida supera el `obtainStockActual` de la BD (RN-INV-02).
 - **Resultado Esperado (Validación / Excepción / Retorno):** Se activa la restricción (Pre-Check), bloqueando la transacción para impedir stocks negativos inválidos y retornando `null`.
 - **Estado Actual:** ¡Listo para codificar!
- **TC-06**
 - **Descripción del Escenario:** Stock Insuficiente (Merma): Intento de reportar una `MERMA` de un producto cuya cantidad supera el inventario físico actual disponible.
 - **Resultado Esperado (Validación / Excepción / Retorno):** El método `validarDisponibilidad` retorna `false`, se imprime el aviso de error en la consola del sistema y el flujo principal se interrumpe retornando `null`.
 - **Estado Actual:** ¡Listo para codificar!
- **TC-07**
 - **Descripción del Escenario:** Inversión de Signo Automatizada: Verificar que al registrar un movimiento de tipo `MERMA`, `SALIDA_VENTA` o `AJUSTE` (si resta) la cantidad se multiplique por `-1`.
 - **Resultado Esperado (Validación / Excepción / Retorno):** Se valida mediante un captor de argumentos (`ArgumentCaptor`) que el valor enviado al repositorio `actualizarStockFisico` sea un entero negativo, garantizando el descuento físico real.
 - **Estado Actual:** ¡Listo para codificar!
- **TC-08**
 - **Descripción del Escenario:** Persistencia Inmutable del Historial: Verificar que, a pesar de que el stock se actualice con un signo negativo, el historial se guarde con la cantidad absoluta en positivo.
 - **Resultado Esperado (Validación / Excepción / Retorno):** Se comprueba mediante aserciones que el objeto enviado a `registrarMovimiento` contenga el valor positivo absoluto (`Math.abs`) para una correcta lectura de auditoría.
 - **Estado Actual:** ¡Listo para codificar!
- **TC-09**
 - **Descripción del Escenario:** Fallo en Actualización de Stock: Escenario donde el repositorio `actualizarStockFisico` retorna `false` por un bloqueo o error en la tabla de la base de datos.
 - **Resultado Esperado (Validación / Excepción / Retorno):** El servicio detecta el fallo de persistencia, aborta la operación imprimiendo un mensaje en `System.err` y retorna `null` sin registrar el historial.

- **Estado Actual:** ¡Listo para codificar!
- **TC-10**
 - **Descripción del Escenario:** Fallo en Registro de Historial: Escenario donde el stock se descuenta con éxito pero falla el insert final en la tabla `historial_inventario`.
 - **Resultado Esperado (Validación / Excepción / Retorno):** El método `movimientoRepo.registrarMovimiento` retorna `false`, el servicio intercepta el error de consistencia y devuelve un valor `null`.
 - **Estado Actual:** ¡Listo para codificar!
- **TC-11**
 - **Descripción del Escenario:** Disparador de Alerta Stock Crítico: Procesamiento exitoso de un movimiento que reduce las unidades por debajo del límite mínimo parametrizado.
 - **Resultado Esperado (Validación / Excepción / Retorno):** El flujo finaliza correctamente y se verifica mediante Mockito que el método de negocio `movimientoRepo.generarAlertaStock()` sea invocado exactamente una vez (`times(1)`).
 - **Estado Actual:** ¡Listo para codificar!
- **TC-12**
 - **Descripción del Escenario:** Fallo Catastrófico de Conexión (SQL): Simulación de una excepción en tiempo de ejecución (`RuntimeException`) durante el acceso a la base de datos SQLite.
 - **Resultado Esperado (Validación / Excepción / Retorno):** El bloque general `catch (Exception e)` captura el error, imprime la traza con `e.printStackTrace()` para depuración y el sistema retorna un valor seguro de `null`.
 - **Estado Actual:** ¡Listo para codificar!